
Introduction to Distributed Systems

Computer systems are undergoing a revolution. From 1945, when the modern computer era began, until about 1985, computers were large and expensive. Even minicomputers normally cost tens of thousands of dollars each. As a result, most organizations had only a handful of computers, and for lack of a way to connect them, these operated independently from one another.

Starting in the mid-1980s, however, two advances in technology began to change that situation. The first was the development of powerful microprocessors. Initially, these were 8-bit machines, but soon 16-, 32-, and even 64-bit CPUs became common. Many of these had the computing power of a decent-sized mainframe (i.e., large) computer, but for a fraction of the price.

The amount of improvement that has occurred in computer technology in the past half century is truly staggering and totally unprecedented in other industries. From a machine that cost 10 million dollars and executed 1 instruction per second, we have come to machines that cost 1000 dollars and execute 10 million instructions per second, a price/performance gain of 10^{11} . If cars had improved at this rate in the same time period, a Rolls Royce would now cost 10 dollars and get a billion miles per gallon. (Unfortunately, it would probably also have a 200-page manual telling how to open the door.)

The second development was the invention of high-speed computer networks. The **local area networks** or **LANs** allow dozens, or even hundreds, of machines within a building to be connected in such a way that small amounts of

information can be transferred between machines in a millisecond or so. Larger amounts of data can be moved between machines at rates of 10 to 100 million bits/sec and sometimes more. The **wide area networks** or **WANs** allow millions of machines all over the earth to be connected at speeds varying from 64 Kbps (kilobits per second) to gigabits per second for some advanced experimental networks.

The result of these technologies is that it is now not only feasible, but easy, to put together computing systems composed of large numbers of CPUs connected by a high-speed network. They are usually called **distributed systems**, in contrast to the previous **centralized systems** (or **single-processor systems**) consisting of a single CPU, its memory, peripherals, and some terminals.

There is only one fly in the ointment: software. Distributed systems need radically different software than centralized systems do. In particular, the necessary operating systems are only beginning to emerge. The first few steps have been taken, but there is still a long way to go. Nevertheless, enough is already known about these distributed operating systems that we can present the basic ideas. The rest of this book is devoted to studying concepts, implementation, and examples of distributed operating systems.

1.1. WHAT IS A DISTRIBUTED SYSTEM?

Various definitions of distributed systems have been given in the literature, none of them satisfactory and none of them in agreement with any of the others. For our purposes it is sufficient to give a loose characterization:

A distributed system is a collection of independent computers that appear to the users of the system as a single computer.

This definition has two aspects. The first one deals with hardware: the machines are autonomous. The second one deals with software: the users think of the system as a single computer. Both are essential. We will come back to these points later in this chapter, after going over some background material on both the hardware and the software.

Rather than going further with definitions, it is probably more helpful to give several examples of distributed systems. As a first example, consider a network of workstations in a university or company department. In addition to each user's personal workstation, there might be a pool of processors in the machine room that are not assigned to specific users but are allocated dynamically as needed. Such a system might have a single file system, with all files accessible from all machines in the same way and using the same path name. Furthermore, when a user typed a command, the system could look for the best place to execute that command, possibly on the user's own workstation, possibly on an idle

workstation belonging to someone else, and possibly on one of the unassigned processors in the machine room. If the system as a whole looked and acted like a classical single-processor timesharing system, it would qualify as a distributed system.

As a second example, consider a factory full of robots, each containing a powerful computer for handling vision, planning, communication, and other tasks. When a robot on the assembly line notices that a part it is supposed to install is defective, it asks another robot in the parts department to bring it a replacement. If all the robots act like peripheral devices attached to the same central computer and the system can be programmed that way, it too counts as a distributed system.

As a final example, think about a large bank with hundreds of branch offices all over the world. Each office has a master computer to store local accounts and handle local transactions. In addition, each computer has the ability to talk to all other branch computers and with a central computer at headquarters. If transactions can be done without regard to where a customer or account is, and the users do not notice any difference between this system and the old centralized mainframe that it replaced, it too would be considered a distributed system.

1.2. GOALS

Just because it is possible to build distributed systems does not necessarily mean that it is a good idea. After all, with current technology it is possible to put four floppy disk drives on a personal computer. It is just that doing so would be pointless. In this section we will discuss the motivation and goals of typical distributed systems and look at their advantages and disadvantages compared to traditional centralized systems.

1.2.1. Advantages of Distributed Systems over Centralized Systems

The real driving force behind the trend toward decentralization is economics. A quarter of a century ago, computer pundit and gadfly Herb Grosch stated what later came to be known as Grosch's law: The computing power of a CPU is proportional to the square of its price. By paying twice as much, you could get four times the performance. This observation fit the mainframe technology of its time quite well, and led most organizations to buy the largest single machine they could afford.

With microprocessor technology, Grosch's law no longer holds. For a few hundred dollars you can get a CPU chip that can execute more instructions per second than one of the largest 1980s mainframes. If you are willing to pay twice as much, you get the same CPU, but running at a somewhat higher clock speed.

As a result, the most cost-effective solution is frequently to harness a large number of cheap CPUs together in a system. Thus the leading reason for the trend toward distributed systems is that these systems potentially have a much better price/performance ratio than a single large centralized system would have. In effect, a distributed system gives more bang for the buck.

A slight variation on this theme is the observation that a collection of microprocessors cannot only give a better price/performance ratio than a single mainframe, but may yield an absolute performance that no mainframe can achieve at any price. For example, with current technology it is possible to build a system from 10,000 modern CPU chips, each of which runs at 50 MIPS (Millions of Instructions Per Second), for a total performance of 500,000 MIPS. For a single processor (i.e., CPU) to achieve this, it would have to execute an instruction in 0.002 nsec (2 picosec). No existing machine even comes close to this, and both theoretical and engineering considerations make it unlikely that any machine ever will. Theoretically, Einstein's theory of relativity dictates that nothing can travel faster than light, which can cover only 0.6 mm in 2 picosec. Practically, a computer of that speed fully contained in a 0.6-mm cube would generate so much heat that it would melt instantly. Thus whether the goal is normal performance at low cost or extremely high performance at greater cost, distributed systems have much to offer.

As an aside, some authors make a distinction between *distributed systems*, which are designed to allow many users to work together, and *parallel systems*, whose only goal is to achieve maximum speedup on a single problem, as our 500,000-MIPS machine might. We believe that this distinction is difficult to maintain because the design spectrum is really a continuum. We prefer to use the term "distributed system" in the broadest sense to denote any system in which multiple interconnected CPUs work together.

A next reason for building a distributed system is that some applications are inherently distributed. A supermarket chain might have many stores, each of which gets goods delivered locally (possibly from local farms), makes local sales, and makes local decisions about which vegetables are so old or rotten that they must be thrown out. It therefore makes sense to keep track of inventory at each store on a local computer rather than centrally at corporate headquarters. After all, most queries and updates will be done locally. Nevertheless, from time to time, top management may want to find out how many rutabagas it currently owns. One way to accomplish this goal is to make the complete system look like a single computer to the application programs, but implement decentrally, with one computer per store as we have described. This would then be a commercial distributed system.

Another inherently distributed system is what is often called **computer-supported cooperative work**, in which a group of people, located far from each other, are working together, for example, to produce a joint report. Given the

long term trends in the computer industry, one can easily imagine a whole new area, **computer-supported cooperative games**, in which players at different locations play against each other in real time. One can imagine electronic hide-and-seek in a big multidimensional maze, and even electronic dogfights with each player using a local flight simulator to try to shoot down the other players, with each player's screen showing the view out of the player's plane, including other planes that fly within visual range.

Another potential advantage of a distributed system over a centralized system is higher reliability. By distributing the workload over many machines, a single chip failure will bring down at most one machine, leaving the rest intact. Ideally, if 5 percent of the machines are down at any moment, the system should be able to continue to work with a 5 percent loss in performance. For critical applications, such as control of nuclear reactors or aircraft, using a distributed system to achieve high reliability may be the dominant consideration.

Finally, incremental growth is also potentially a big plus. Often, a company will buy a mainframe with the intention of doing all its work on it. If the company prospers and the workload grows, at a certain point the mainframe will no longer be adequate. The only solutions are either to replace the mainframe with a larger one (if it exists) or to add a second mainframe. Both of these can wreak major havoc on the company's operations. In contrast, with a distributed system, it may be possible simply to add more processors to the system, thus allowing it to expand gradually as the need arises. These advantages are summarized in Fig. 1-1.

Item	Description
Economics	Microprocessors offer a better price/performance than mainframes
Speed	A distributed system may have more total computing power than a mainframe
Inherent distribution	Some applications involve spatially separated machines
Reliability	If one machine crashes, the system as a whole can still survive
Incremental growth	Computing power can be added in small increments

Fig. 1-1. Advantages of distributed systems over centralized systems.

In the long term, the main driving force will be the existence of large numbers of personal computers and the need for people to work together and share information in a convenient way without being bothered by geography or the physical distribution of people, data, and machines.

1.2.2. Advantages of Distributed Systems over Independent PCs

Given that microprocessors are a cost-effective way to do business, why not just give everyone his[†] own PC and let people work independently? For one thing, many users need to share data. For example, airline reservation clerks need access to the master data base of flights and existing reservations. Giving each clerk his own private copy of the entire data base would not work, since nobody would know which seats the other clerks had already sold. Shared data are absolutely essential to this and many other applications, so the machines must be interconnected. Interconnecting the machines leads to a distributed system.

Sharing often involves more than just data. Expensive peripherals, such as color laser printers, phototypesetters, and massive archival storage devices (e.g., optical jukeboxes), are also candidates.

A third reason to connect a group of isolated computers into a distributed system is to achieve enhanced person-to-person communication. For many people, electronic mail has numerous attractions over paper mail, telephone, and FAX. It is much faster than paper mail, does not require both parties to be available at the same time as does the telephone, and unlike FAX, produces documents that can be edited, rearranged, stored in the computer, and manipulated with text processing programs.

Finally, a distributed system is potentially more flexible than giving each user an isolated personal computer. Although one model is to give each person a personal computer and connect them all with a LAN, this is not the only possibility. Another one is to have a mixture of personal and shared computers, perhaps of different sizes, and let jobs run on the most appropriate one, rather than always on the owner's machine. In this way, the workload can be spread over the computers more effectively, and the loss of a few machines may be compensated for by letting people run their jobs elsewhere. Figure 1-2 summarizes these points.

1.2.3. Disadvantages of Distributed Systems

Although distributed systems have their strengths, they also have their weaknesses. In this section, we will point out a few of them. We have already hinted at the worst problem: software. With the current state-of-the-art, we do not have much experience in designing, implementing, and using distributed software. What kinds of operating systems, programming languages, and applications are appropriate for these systems? How much should the users know

[†] Please read "his" as "his or hers" throughout this book.

Item	Description
Data sharing	Allow many users access to a common data base
Device sharing	Allow many users to share expensive peripherals like color printers
Communication	Make human-to-human communication easier, for example, by electronic mail
Flexibility	Spread the workload over the available machines in the most cost effective way

Fig. 1-2. Advantages of distributed systems over isolated (personal) computers.

about the distribution? How much should the system do and how much should the users do? The experts differ (not that this is unusual with experts, but when it comes to distributed systems, they are barely on speaking terms). As more research is done, this problem will diminish, but for the moment it should not be underestimated.

A second potential problem is due to the communication network. It can lose messages, which requires special software to be able to recover, and it can become overloaded. When the network saturates, it must either be replaced or a second one must be added. In both cases, some portion of one or more buildings may have to be rewired at great expense, or network interface boards may have to be replaced (e.g., by fiber optics). Once the system comes to depend on the network, its loss or saturation can negate most of the advantages the distributed system was built to achieve.

Finally, the easy sharing of data, which we described above as an advantage, may turn out to be a two-edged sword. If people can conveniently access data all over the system, they may equally be able to conveniently access data that they have no business looking at. In other words, security is often a problem. For data that must be kept secret at all costs, it is often preferable to have a dedicated, isolated personal computer that has no network connections to any other machines, and is kept in a locked room with a secure safe in which all the floppy disks are stored. The disadvantages of distributed systems are summarized in Fig. 1-3.

Despite these potential problems, many people feel that the advantages outweigh the disadvantages, and it is expected that distributed systems will become increasingly important in the coming years. In fact, it is likely that within a few years, most organizations will connect most of their computers into large distributed systems to provide better, cheaper, and more convenient service for the users. An isolated computer in a medium-sized or large business or other organization will probably not even exist in ten years.

Item	Description
Software	Little software exists at present for distributed systems
Networking	The network can saturate or cause other problems
Security	Easy access also applies to secret data

Fig. 1-3. Disadvantages of distributed systems.

1.3. HARDWARE CONCEPTS

Even though all distributed systems consist of multiple CPUs, there are several different ways the hardware can be organized, especially in terms of how they are interconnected and how they communicate. In this section we will take a brief look at distributed system hardware, in particular, how the machines are connected together. In the next section we will examine some of the software issues related to distributed systems.

Various classification schemes for multiple CPU computer systems have been proposed over the years, but none of them have really caught on and been widely adopted. Probably the most frequently cited taxonomy is Flynn's (1972), although it is fairly rudimentary. Flynn picked two characteristics that he considered essential: the number of instruction streams and the number of data streams. A computer with a single instruction stream and a single data stream is called SISD. All traditional uniprocessor computers (i.e., those having only one CPU) fall in this category, from personal computers to large mainframes.

The next category is SIMD, single instruction stream, multiple data stream. This type refers to array processors with one instruction unit that fetches an instruction, and then commands many data units to carry it out in parallel, each with its own data. These machines are useful for computations that repeat the same calculation on many sets of data, for example, adding up all the elements of 64 independent vectors. Some supercomputers are SIMD.

The next category is MISD, multiple instruction stream, single data stream. No known computers fit this model. Finally, comes MIMD, which essentially means a group of independent computers, each with its own program counter, program, and data. All distributed systems are MIMD, so this classification system is not tremendously useful for our purposes.

Although Flynn stopped here, we will go further. In Fig. 1-4, we divide all MIMD computers into two groups: those that have shared memory, usually called **multiprocessors**, and those that do not, sometimes called **multicomputers**. The essential difference is this: in a multiprocessor, there is a single virtual

address space that is shared by all CPUs. If any CPU writes, for example, the value 44 to address 1000, any other CPU subsequently reading from *its* address 1000 will get the value 44. All the machines share the same memory.

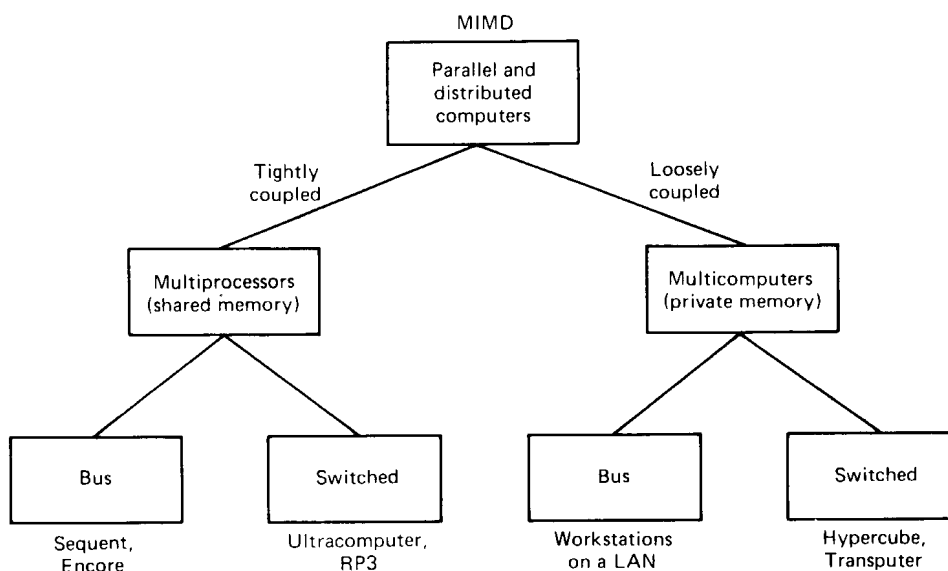


Fig. 1-4. A taxonomy of parallel and distributed computer systems.

In contrast, in a multicomputer, every machine has its own private memory. If one CPU writes the value 44 to address 1000, when another CPU reads address 1000 it will get whatever value was there before. The write of 44 does not affect *its* memory at all. A common example of a multicomputer is a collection of personal computers connected by a network.

Each of these categories can be further divided based on the architecture of the interconnection network. In Fig. 1-4 we describe these two categories as **bus** and **switched**. By bus we mean that there is a single network, backplane, bus, cable, or other medium that connects all the machines. Cable television uses a scheme like this: the cable company runs a wire down the street, and all the subscribers have taps running to it from their television sets.

Switched systems do not have a single backbone like cable television. Instead, there are individual wires from machine to machine, with many different wiring patterns in use. Messages move along the wires, with an explicit switching decision made at each step to route the message along one of the outgoing wires. The worldwide public telephone system is organized in this way.

Another dimension to our taxonomy is that in some systems the machines are **tightly coupled** and in others they are **loosely coupled**. In a tightly-coupled

system, the delay experienced when a message is sent from one computer to another is short, and the data rate is high; that is, the number of bits per second that can be transferred is large. In a loosely-coupled system, the opposite is true: the intermachine message delay is large and the data rate is low. For example, two CPU chips on the same printed circuit board and connected by wires etched onto the board are likely to be tightly coupled, whereas two computers connected by a 2400 bit/sec modem over the telephone system are certain to be loosely coupled.

Tightly-coupled systems tend to be used more as parallel systems (working on a single problem) and loosely-coupled ones tend to be used as distributed systems (working on many unrelated problems), although this is not always true. One famous counterexample is a project in which hundreds of computers all over the world worked together trying to factor a huge number (about 100 digits). Each computer was assigned a different range of divisors to try, and they all worked on the problem in their spare time, reporting the results back by electronic mail when they finished.

On the whole, multiprocessors tend to be more tightly coupled than multicomputers, because they can exchange data at memory speeds, but some fiber-optic based multicomputers can also work at memory speeds. Despite the vagueness of the terms “tightly coupled” and “loosely coupled,” they are useful concepts, just as saying “Jack is fat and Jill is thin” conveys information about girth even though one can get into a fair amount of discussion about the concepts of “fatness” and “thinness.”

In the following four sections, we will look at the four categories of Fig. 1-4 in more detail, namely bus multiprocessors, switched multiprocessors, bus multicomputers, and switched multicomputers. Although these topics are not directly related to our main concern, distributed operating systems, they will shed some light on the subject because as we shall see, different categories of machines use different kinds of operating systems.

1.3.1. Bus-Based Multiprocessors

Bus-based multiprocessors consist of some number of CPUs all connected to a common bus, along with a memory module. A simple configuration is to have a high-speed backplane or motherboard into which CPU and memory cards can be inserted. A typical bus has 32 or 64 address lines, 32 or 64 data lines, and perhaps 32 or more control lines, all of which operate in parallel. To read a word of memory, a CPU puts the address of the word it wants on the bus address lines, then puts a signal on the appropriate control lines to indicate that it wants to read. The memory responds by putting the value of the word on the data lines to allow the requesting CPU to read it in. Writes work in a similar way.

Since there is only one memory, if CPU A writes a word to memory and

then CPU *B* reads that word back a microsecond later, *B* will get the value just written. A memory that has this property is said to be **coherent**. Coherence plays an important role in distributed operating systems in a variety of ways that we will study later.

The problem with this scheme is that with as few as 4 or 5 CPUs, the bus will usually be overloaded and performance will drop drastically. The solution is to add a high-speed **cache memory** between the CPU and the bus, as shown in Fig. 1-5. The cache holds the most recently accessed words. All memory requests go through the cache. If the word requested is in the cache, the cache itself responds to the CPU, and no bus request is made. If the cache is large enough, the probability of success, called the **hit rate**, will be high, and the amount of bus traffic per CPU will drop dramatically, allowing many more CPUs in the system. Cache sizes of 64K to 1M are common, which often gives a hit rate of 90 percent or more.

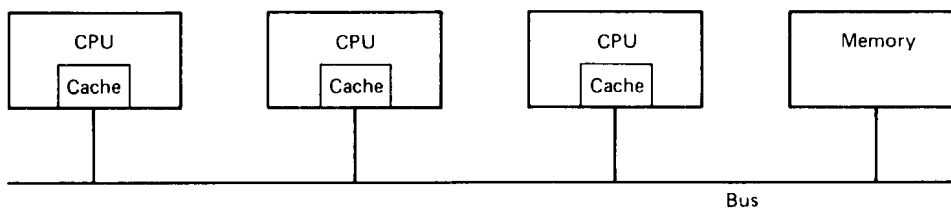


Fig. 1-5. A bus-based multiprocessor.

However, the introduction of caches also brings a serious problem with it. Suppose that two CPUs, *A* and *B*, each read the same word into their respective caches. Then *A* overwrites the word. When *B* next reads that word, it gets the old value from its cache, not the value *A* just wrote. The memory is now incoherent, and the system is difficult to program.

Many researchers have studied this problem, and various solutions are known. Below we will sketch one of them. Suppose that the cache memories are designed so that whenever a word is written to the cache, it is written through to memory as well. Such a cache is, not surprisingly, called a **write-through cache**. In this design, cache hits for reads do not cause bus traffic, but cache misses for reads, and all writes, hits and misses, cause bus traffic.

In addition, all caches constantly monitor the bus. Whenever a cache sees a write occurring to a memory address present in its cache, it either removes that entry from its cache, or updates the cache entry with the new value. Such a cache is called a **snoopy cache** (or sometimes, a **snooping cache**) because it is always “snooping” (eavesdropping) on the bus. A design consisting of snoopy write-through caches is coherent and is invisible to the programmer. Nearly all bus-based multiprocessors use either this architecture or one closely related to it.

Using it, it is possible to put about 32 or possibly 64 CPUs on a single bus. For more about bus-based multiprocessors, see Lilja (1993).

1.3.2. Switched Multiprocessors

To build a multiprocessor with more than 64 processors, a different method is needed to connect the CPUs with the memory. One possibility is to divide the memory up into modules and connect them to the CPUs with a **crossbar switch**, as shown in Fig. 1-6(a). Each CPU and each memory has a connection coming out of it, as shown. At every intersection is a tiny electronic **crosspoint switch** that can be opened and closed in hardware. When a CPU wants to access a particular memory, the crosspoint switch connecting them is closed momentarily, to allow the access to take place. The virtue of the crossbar switch is that many CPUs can be accessing memory at the same time, although if two CPUs try to access the same memory simultaneously, one of them will have to wait.

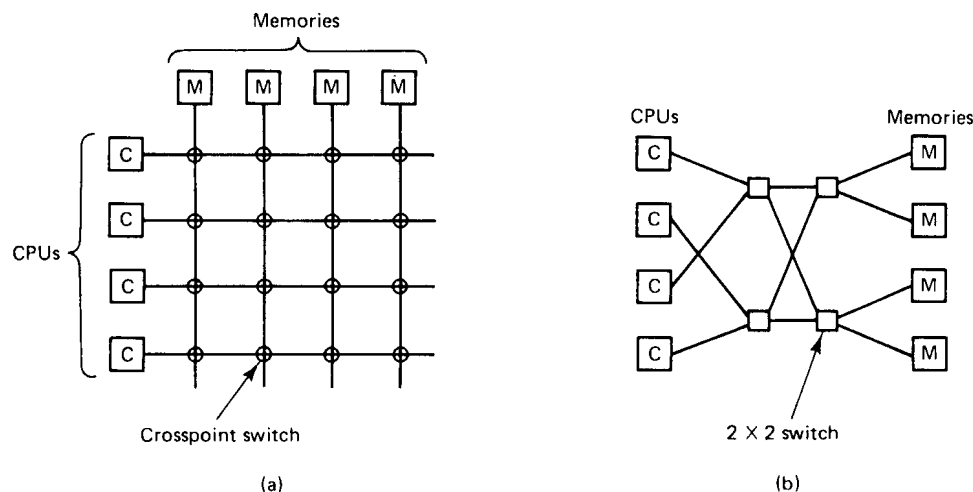


Fig. 1-6. (a) A crossbar switch. (b) An omega switching network.

The downside of the crossbar switch is that with n CPUs and n memories, n^2 crosspoint switches are needed. For large n , this number can be prohibitive. As a result, people have looked for, and found, alternative switching networks that require fewer switches. The **omega network** of Fig. 1-6(b) is one example. This network contains four 2×2 switches, each having two inputs and two outputs. Each switch can route either input to either output. A careful look at the figure will show that with proper settings of the switches, every CPU can access every memory. These switches can be set in nanoseconds or less.

In the general case, with n CPUs and n memories, the omega network requires $\log_2 n$ switching stages, each containing $n/2$ switches, for a total of $(n \log_2 n)/2$ switches. Although for large n this is much better than n^2 , it is still substantial.

Furthermore, there is another problem: delay. For example, for $n = 1024$, there are 10 switching stages from the CPU to the memory, and another 10 for the word requested to come back. Suppose that the CPU is a modern RISC chip running at 100 MIPS; that is, the instruction execution time is 10 nsec. If a memory request is to traverse a total of 20 switching stages (10 outbound and 10 back) in 10 nsec, the switching time must be 500 picosec (0.5 nsec). The complete multiprocessor will need 5120 500-picosec switches. This is not going to be cheap.

People have attempted to reduce the cost by going to hierarchical systems. Some memory is associated with each CPU. Each CPU can access its own local memory quickly, but accessing anybody else's memory is slower. This design gives rise to what is known as a **NUMA (NonUniform Memory Access)** machine. Although NUMA machines have better average access times than machines based on omega networks, they have the new complication that the placement of the programs and data becomes critical in order to make most access go to the local memory.

To summarize, bus-based multiprocessors, even with snoopy caches, are limited by the amount of bus capacity to about 64 CPUs at most. To go beyond that requires a switching network, such as a crossbar switch, an omega switching network, or something similar. Large crossbar switches are very expensive, and large omega networks are both expensive and slow. NUMA machines require complex algorithms for good software placement. The conclusion is clear: building a large, tightly-coupled, shared memory multiprocessor is possible, but is difficult and expensive.

1.3.3. Bus-Based Multicomputers

On the other hand, building a multicomputer (i.e., no shared memory) is easy. Each CPU has a direct connection to its own local memory. The only problem left is how the CPUs communicate with each other. Clearly, some interconnection scheme is needed here, too, but since it is only for CPU-to-CPU communication, the volume of traffic will be several orders of magnitude lower than when the interconnection network is also used for CPU-to-memory traffic.

In Fig. 1-7 we see a bus-based multicomputer. It looks topologically similar to the bus-based multiprocessor, but since there will be much less traffic over it, it need not be a high-speed backplane bus. In fact, it can be a much lower speed LAN (typically, 10–100 Mbps, compared to 300 Mbps and up for a backplane bus). Thus Fig. 1-7 is more often a collection of workstations on a LAN than a

collection of CPU cards inserted into a fast bus (although the latter configuration is definitely a possible design).

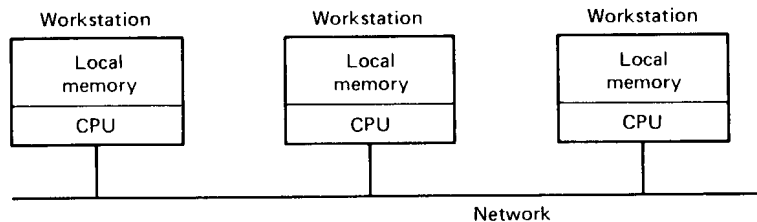


Fig. 1-7. A multicomputer consisting of workstations on a LAN.

1.3.4. Switched Multicomputers

Our last category consists of switched multicomputers. Various interconnection networks have been proposed and built, but all have the property that each CPU has direct and exclusive access to its own, private memory. Figure 1-8 shows two popular topologies, a grid and a hypercube. Grids are easy to understand and lay out on printed circuit boards. They are best suited to problems that have an inherent two-dimensional nature, such as graph theory or vision (e.g., robot eyes or analyzing photographs).

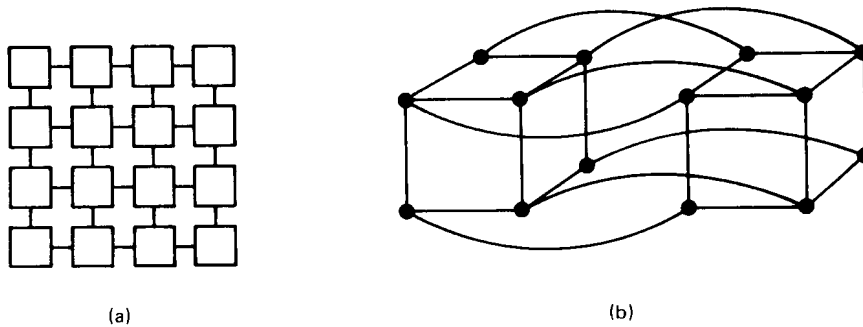


Fig. 1-8. (a) Grid. (b) Hypercube.

A **hypercube** is an n -dimensional cube. The hypercube of Fig. 1-8(b) is four-dimensional. It can be thought of as two ordinary cubes, each with 8 vertices and 12 edges. Each vertex is a CPU. Each edge is a connection between two CPUs. The corresponding vertices in each of the two cubes are connected.

To expand the hypercube to five dimensions, we would add another set of two interconnected cubes to the figure, connect the corresponding edges in the

two halves, and so on. For an n -dimensional hypercube, each CPU has n connections to other CPUs. Thus the complexity of the wiring increases only logarithmically with the size. Since only nearest neighbors are connected, many messages have to make several hops to reach their destination. However, the longest possible path also grows logarithmically with the size, in contrast to the grid, where it grows as the square root of the number of CPUs. Hypercubes with 1024 CPUs have been commercially available for several years, and hypercubes with as many as 16,384 CPUs are starting to become available.

1.4. SOFTWARE CONCEPTS

Although the hardware is important, the software is even more important. The image that a system presents to its users, and how they think about the system, is largely determined by the operating system software, not the hardware. In this section we will introduce the various types of operating systems for the multiprocessors and multicomputers we have just studied, and discuss which kind of software goes with which kind of hardware.

Operating systems cannot be put into nice, neat pigeonholes like hardware. By nature software is vague and amorphous. Still, it is more-or-less possible to distinguish two kinds of operating systems for multiple CPU systems: loosely coupled and tightly coupled. As we shall see, loosely and tightly-coupled software is roughly analogous to loosely and tightly-coupled hardware.

Loosely-coupled software allows machines and users of a distributed system to be fundamentally independent of one another, but still to interact to a limited degree where that is necessary. Consider a group of personal computers, each of which has its own CPU, its own memory, its own hard disk, and its own operating system, but which share some resources, such as laser printers and data bases, over a LAN. This system is loosely coupled, since the individual machines are clearly distinguishable, each with its own job to do. If the network should go down for some reason, the individual machines can still continue to run to a considerable degree, although some functionality may be lost (e.g., the ability to print files).

To show how difficult it is to make definitions in this area, now consider the same system as above, but without the network. To print a file, the user writes the file on a floppy disk, carries it to the machine with the printer, reads it in, and then prints it. Is this still a distributed system, only now even more loosely coupled? It's hard to say. From a fundamental point of view, there is not really any theoretical difference between communicating over a LAN and communicating by carrying floppy disks around. At most one can say that the delay and data rate are worse in the second example.

At the other extreme we might find a multiprocessor dedicated to running a

single chess program in parallel. Each CPU is assigned a board to evaluate, and it spends its time examining that board and all the boards that can be generated from it. When the evaluation is finished, the CPU reports back the results and is given a new board to work on. The software for this system, both the application program and the operating system required to support it, is clearly much more tightly coupled than in our previous example.

We have now seen four kinds of distributed hardware and two kinds of distributed software. In theory, there should be eight combinations of hardware and software. In fact, only four are worth distinguishing, because to the user, the interconnection technology is not visible. For most purposes, a multiprocessor is a multiprocessor, whether it uses a bus with snoopy caches or uses an omega network. In the following sections we will look at some of the most common combinations of hardware and software.

1.4.1. Network Operating Systems

Let us start with loosely-coupled software on loosely-coupled hardware, since this is probably the most common combination at many organizations. A typical example is a network of workstations connected by a LAN. In this model, each user has a workstation for his exclusive use. It may or may not have a hard disk. It definitely has its own operating system. All commands are normally run locally, right on the workstation.

However, it is sometimes possible for a user to log into another workstation remotely by using a command such as

```
rlogin machine
```

The effect of this command is to turn the user's own workstation into a remote terminal logged into the remote machine. Commands typed on the keyboard are sent to the remote machine, and output from the remote machine is displayed on the screen. To switch to a different remote machine, it is necessary first to log out, then to use the *rlogin* command to connect to another machine. At any instant, only one machine can be used, and the selection of the machine is entirely manual.

Networks of workstations often also have a remote copy command to copy files from one machine to another. For example, a command like

```
rcp machine1:file1 machine2:file2
```

might copy the file *file1* from *machine1* to *machine2* and give it the name *file2* there. Again here, the movement of files is explicit and requires the user to be completely aware of where all files are located and where all commands are being executed.

While better than nothing, this form of communication is extremely

primitive and has led system designers to search for more convenient forms of communication and information sharing. One approach is to provide a shared, global file system accessible from all the workstations. The file system is supported by one or more machines called **file servers**. The file servers accept requests from user programs running on the other (nonserver) machines, called **clients**, to read and write files. Each incoming request is examined and executed, and the reply is sent back, as illustrated in Fig. 1-9.

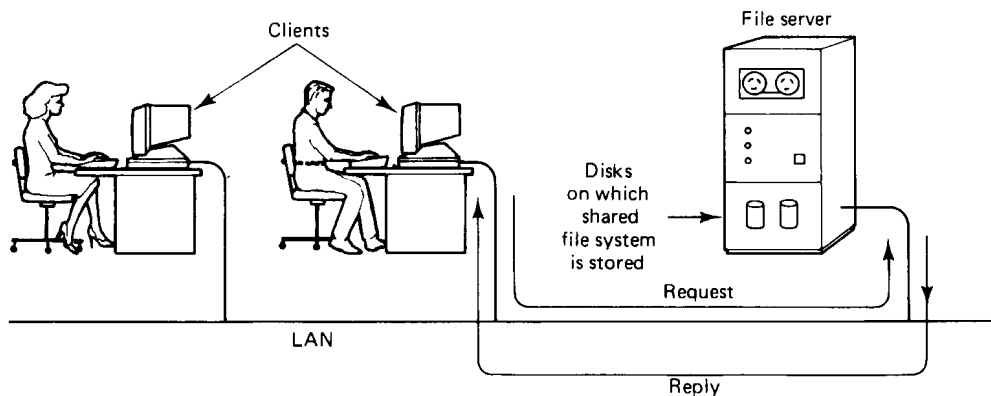


Fig. 1-9. Two clients and a server in a network operating system.

File servers generally maintain hierarchical file systems, each with a root directory containing subdirectories and files. Workstations can import or mount these file systems, augmenting their local file systems with those located on the servers. For example, in Fig. 1-10, two file servers are shown. One has a directory called *games*, while the other has a directory called *work*. These directories each contain several files. Both of the clients shown have mounted both of the servers, but they have mounted them in different places in their respective file systems. Client 1 has mounted them in its root directory, and can access them as */games* and */work*, respectively. Client 2, like client 1, has mounted *games* in its root directory, but regarding the reading of mail and news as a kind of game, has created a directory */games/work* and mounted *work* there. Consequently, it can access *news* using the path */games/work/news* rather than */work/news*.

While it does not matter where a client mounts a server in its directory hierarchy, it is important to notice that different clients can have a different view of the file system. The name of a file depends on where it is being accessed from, and how that machine has set up its file system. Because each workstation operates relatively independently of the others, there is no guarantee that they all present the same directory hierarchy to their programs.

The operating system that is used in this kind of environment must manage the individual workstations and file servers and take care of the communication

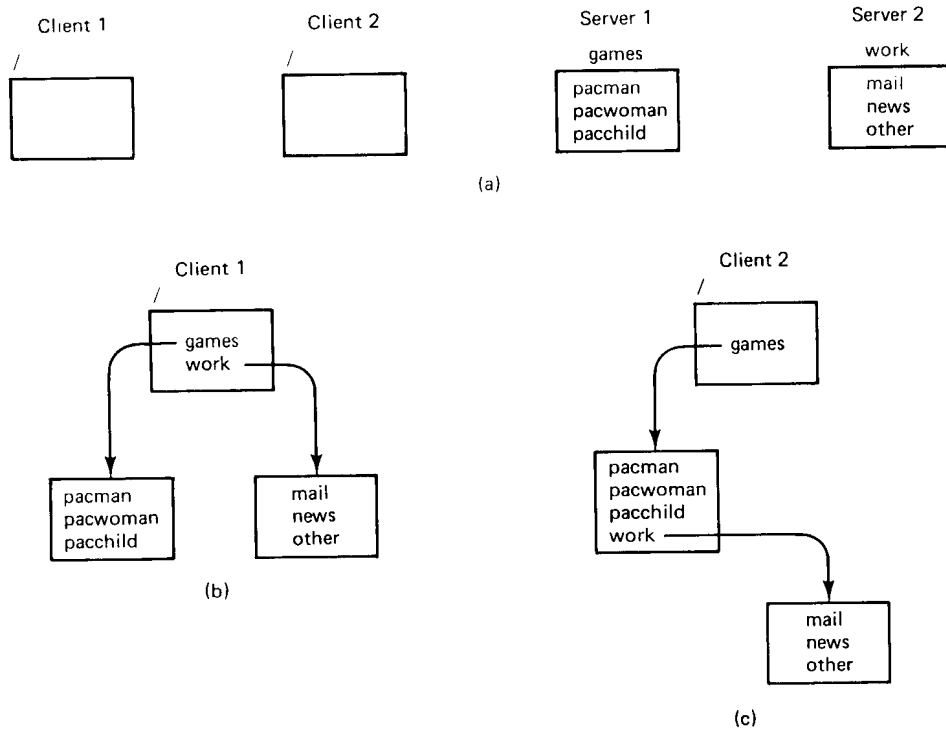


Fig. 1-10. Different clients may mount the servers in different places.

between them. It is possible that the machines all run the same operating system, but this is not required. If the clients and servers run on different systems, as a bare minimum they must agree on the format and meaning of all the messages that they may potentially exchange. In a situation like this, where each machine has a high degree of autonomy and there are few system-wide requirements, people usually speak of a **network operating system**.

1.4.2. True Distributed Systems

Network operating systems are loosely-coupled software on loosely-coupled hardware. Other than the shared file system, it is quite apparent to the users that such a system consists of numerous computers. Each can run its own operating system and do whatever its owner wants. There is essentially no coordination at all, except for the rule that client-server traffic must obey the system's protocols.

The next evolutionary step beyond this is tightly-coupled software on the same loosely-coupled (i.e., multicomputer) hardware. The goal of such a system is to create the illusion in the minds of the users that the entire network of

computers is a single timesharing system, rather than a collection of distinct machines. Some authors refer to this property as the **single-system image**. Others put it slightly differently, saying that a distributed system is one that runs on a collection of networked machines but acts like a **virtual uniprocessor**. No matter how it is expressed, the essential idea is that the users should not have to be aware of the existence of multiple CPUs in the system. No current system fulfills this requirement entirely, but a number of candidates are on the horizon. These will be discussed later in the book.

What are some characteristics of a distributed system? To start with, there must be a single, global interprocess communication mechanism so that any process can talk to any other process. It will not do to have different mechanisms on different machines or different mechanisms for local communication and remote communication. There must also be a global protection scheme. Mixing access control lists, the UNIX[®] protection bits, and capabilities will not give a single system image.

Process management must also be the same everywhere. How processes are created, destroyed, started, and stopped must not vary from machine to machine. In short, the idea behind network operating systems, namely that any machine can do whatever it wants to as long as it obeys the standard protocols when engaging in client-server communication, is not enough. Not only must there be a single set of system calls available on all machines, but these calls must be designed so that they make sense in a distributed environment.

The file system must look the same everywhere, too. Having file names restricted to 11 characters in some locations and being unrestricted in others is undesirable. Also, every file should be visible at every location, subject to protection and security constraints, of course.

As a logical consequence of having the same system call interface everywhere, it is normal that identical kernels run on all the CPUs in the system. Doing so makes it easier to coordinate activities that must be global. For example, when a process has to be started up, all the kernels have to cooperate in finding the best place to execute it. In addition, a global file system is needed.

Nevertheless, each kernel can have considerable control over its own local resources. For example, since there is no shared memory, it is logical to allow each kernel to manage its own memory. For example, if swapping or paging is used, the kernel on each CPU is the logical place to determine what to swap or page. There is no reason to centralize this authority. Similarly, if multiple processes are running on some CPU, it makes sense to do the scheduling right there, too.

A considerable body of knowledge is now available about designing and implementing distributed operating systems. Rather than going into these issues here, we will first finish off our survey of the different combinations of hardware and software, and come back to them in Sec. 1.5.

1.4.3. Multiprocessor Timesharing Systems

The last combination we wish to discuss is tightly-coupled software on tightly-coupled hardware. While various special-purpose machines exist in this category (such as dedicated data base machines), the most common general-purpose examples are multiprocessors that are operated as a UNIX timesharing system, but with multiple CPUs instead of one CPU. To the outside world, a multiprocessor with 32 30-MIPS CPUs acts very much like a single 960-MIPS CPU (this is the single-system image discussed above). Except that implementing it on a multiprocessor makes life much easier, since the entire design can be centralized.

The key characteristic of this class of system is the existence of a single run queue: a list of all the processes in the system that are logically unblocked and ready to run. The run queue is a data structure kept in the shared memory. As an example, consider the system of Fig. 1-11, which has three CPUs and five processes that are ready to run. All five processes are located in the shared memory, and three of them are currently executing: process *A* on CPU 1, process *B* on CPU 2, and process *C* on CPU 3. The other two processes, *D* and *E*, are also in memory, waiting their turn.

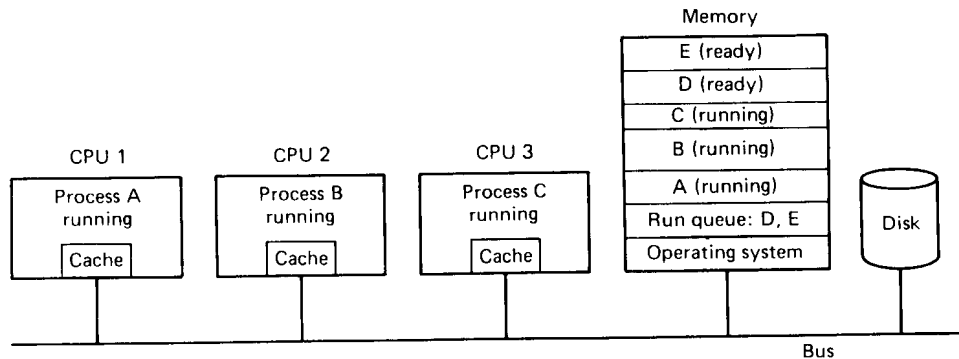


Fig. 1-11. A multiprocessor with a single run queue.

Now suppose that process *B* blocks waiting for I/O or its quantum runs out. Either way, CPU 2 must suspend it, and find another process to run. CPU 2 will normally begin executing operating system code (located in the shared memory). After having saved all of *B*'s registers, it will enter a critical region to run the scheduler to look for another process to run. It is essential that the scheduler be run as a critical region to prevent two CPUs from choosing the same process to run next. The necessary mutual exclusion can be achieved by using monitors, semaphores, or any other standard construction used in singleprocessor systems.

Once CPU 2 has gained exclusive access to the run queue, it can remove the first entry, *D*, exit from the critical region, and begin executing *D*. Initially, execution will be slow, since CPU 2's cache is full of words belonging to that part of the shared memory containing process *B*, but after a little while, these will have been purged and the cache will be full of *D*'s code and data, so execution will speed up.

Because none of the CPUs have local memory and all programs are stored in the global shared memory, it does not matter on which CPU a process runs. If a long-running process is scheduled many times before it completes, on the average, it will spend about the same amount of time running on each CPU. The only factor that has any effect at all on CPU choice is the slight gain in performance when a process starts up on a CPU that is currently caching part of its address space. In other words, if all CPUs are idle, waiting for I/O, and one process becomes ready, it is slightly preferable to allocate it to the CPU it was last using, assuming that no other process has used that CPU since (Vaswani and Zahorjan, 1991).

As an aside, if a process blocks for I/O on a multiprocessor, the operating system has the choice of suspending it or just letting it do busy waiting. If most I/O is completed in less time than it takes to do a process switch, busy waiting is preferable. Some systems let the process keep its processor for a few milliseconds, in the hope that the I/O will complete soon, but if that does not occur before the timer runs out, a process switch is made (Karlin et al., 1991). If most critical regions are short, this approach can avoid many expensive process switches.

An area in which this kind of multiprocessor differs appreciably from a network or distributed system is in the organization of the file system. The operating system normally contains a traditional file system, including a single, unified block cache. When any process executes a system call, a trap is made to the operating system, which carries it out, using semaphores, monitors, or something equivalent, to lock out other CPUs while critical sections are being executed or central tables are being accessed. In this way, when a WRITE system call is done, the central block cache is locked, the new data entered into the cache, and the lock released. Any subsequent READ call will see the new data, just as on a single-processor system. On the whole, the file system is hardly different from a single-processor file system. In fact, on some multiprocessors, one of the CPUs is dedicated to running the operating system; the other ones run user programs. This situation is undesirable, however, as the operating system machine is often a bottleneck. This point is discussed in detail by Boykin and Langerman (1990).

It should be clear that the methods used on the multiprocessor to achieve the appearance of a virtual uniprocessor are not applicable to machines that do not have shared memory. Centralized run queues and block only caches work when all CPUs have access to them with very low delay. Although these data

structures could be simulated on a network of machines, the communication costs make this approach prohibitively expensive.

Figure 1-12 shows some of the differences between the three kinds of systems we have examined above.

Item	Network operating system	Distributed operating system	Multiprocessor operating system
Does it look like a virtual uniprocessor?	No	Yes	Yes
Do all have to run the same operating system?	No	Yes	Yes
How many copies of the operating system are there?	N	N	1
How is communication achieved?	Shared files	Messages	Shared memory
Are agreed upon network protocols required?	Yes	Yes	No
Is there a single run queue?	No	No	Yes
Does file sharing have well-defined semantics?	Usually no	Yes	Yes

Fig. 1-12. Comparison of three different ways of organizing n CPUs.

1.5. DESIGN ISSUES

In the preceding sections we have looked at distributed systems and related topics from both the hardware and software points of view. In the remainder of this chapter we will briefly look at some of the key design issues that people contemplating building a distributed operating system must deal with. We will come back to them in more detail later in the book.

1.5.1. Transparency

Probably the single most important issue is how to achieve the single-system image. In other words, how do the system designers fool everyone into thinking that the collection of machines is simply an old-fashioned timesharing system? A system that realizes this goal is often said to be **transparent**.

Transparency can be achieved at two different levels. Easiest to do is to hide the distribution from the users. For example, when a UNIX user types *make*

to recompile a large number of files in a directory, he need not be told that all the compilations are proceeding in parallel on different machines and are using a variety of file servers to do it. To him, the only thing that is unusual is that the performance of the system is halfway decent for a change. In terms of commands issued from the terminal and results displayed on the terminal, the distributed system can be made to look just like a single-processor system.

At a lower level, it is also possible, but harder, to make the system look transparent to programs. In other words, the system call interface can be designed so that the existence of multiple processors is not visible. Pulling the wool over the programmer's eyes is harder than pulling the wool over the terminal user's eyes, however.

What does transparency really mean? It is one of those slippery concepts that sounds reasonable but is more subtle than it at first appears. As an example, imagine a distributed system consisting of workstations each running some standard operating system. Normally, system services (e.g., reading files) are obtained by issuing a system call that traps to the kernel. In such a system, remote files should be accessed the same way. A system in which remote files are accessed by explicitly setting up a network connection to a remote server and then sending messages to it is not transparent because remote services are then being accessed differently than local ones. The programmer can tell that multiple machines are involved, and this is not allowed.

The concept of transparency can be applied to several aspects of a distributed system, as shown in Fig. 1-13. **Location transparency** refers to the fact that in a true distributed system, users cannot tell where hardware and software resources such as CPUs, printers, files, and data bases are located. The name of the resource must not secretly encode the location of the resource, so names like *machine1:prog.c* or */machine1/prog.c* are not acceptable.

Kind	Meaning
Location transparency	The users cannot tell where resources are located
Migration transparency	Resources can move at will without changing their names
Replication transparency	The users cannot tell how many copies exist
Concurrency transparency	Multiple users can share resources automatically
Parallelism transparency	Activities can happen in parallel without users knowing

Fig. 1-13. Different kinds of transparency in a distributed system.

Migration transparency means that resources must be free to move from one location to another without having their names change. In the example of

Fig. 1-10 we saw how server directories could be mounted in arbitrary places in the clients' directory hierarchy. Since a path like */work/news* does not reveal the location of the server, it is location transparent. However, now suppose that the folks running the servers decide that reading network news really falls in the category "games" rather than in the category "work." Accordingly, they move *news* from server 2 to server 1. The next time client 1 boots and mounts the servers in his customary way, he will notice that */work/news* no longer exists. Instead, there is a new entry, */games/news*. Thus the mere fact that a file or directory has migrated from one server to another has forced it to acquire a new name because the system of remote mounts is not migration transparent.

If a distributed system has **replication transparency**, the operating system is free to make additional copies of files and other resources on its own without the users noticing. Clearly, in the previous example, automatic replication is impossible because the names and locations are so closely tied together. To see how replication transparency might be achievable, consider a collection of n servers logically connected to form a ring. Each server maintains the entire directory tree structure but holds only a subset of the files themselves. To read a file, a client sends a message containing the full path name to any of the servers. That server checks to see if it has the file. If so, it returns the data requested. If not, it forwards the request to the next server in the ring, which then repeats the algorithm. In this system, the servers can decide by themselves to replicate any file on any or all servers, without the users having to know about it. Such a scheme is replication transparent because it allows the system to make copies of heavily used files without the users even being aware that this is happening.

Distributed systems usually have multiple, independent users. What should the system do when two or more users try to access the same resource at the same time? For example, what happens if two users try to update the same file at the same time? If the system is **concurrency transparent**, the users will not notice the existence of other users. One mechanism for achieving this form of transparency would be for the system to lock a resource automatically once someone had started to use it, unlocking it only when the access was finished. In this manner, all resources would only be accessed sequentially, never concurrently.

Finally, we come to the hardest one, **parallelism transparency**. In principle, a distributed system is supposed to appear to the users as a traditional, uniprocessor timesharing system. What happens if a programmer knows that his distributed system has 1000 CPUs and he wants to use a substantial fraction of them for a chess program that evaluates boards in parallel? The theoretical answer is that together the compiler, runtime system, and operating system should be able to figure out how to take advantage of this potential parallelism without the programmer even knowing it. Unfortunately, the current state-of-the-art is nowhere near allowing this to happen. Programmers who actually

want to use multiple CPUs for a single problem will have to program this explicitly, at least for the foreseeable future. Parallelism transparency can be regarded as the holy grail for distributed systems designers. When that has been achieved, the work will have been completed, and it will be time to move on to new fields.

All this notwithstanding, there are times when users do *not* want complete transparency. For example, when a user asks to print a document, he often prefers to have the output appear on the local printer, not one 1000 km away, even if the distant printer is fast, inexpensive, can handle color and smell, and is currently idle.

1.5.2. Flexibility

The second key design issue is flexibility. It is important that the system be flexible because we are just beginning to learn about how to build distributed systems. It is likely that this process will incur many false starts and considerable backtracking. Design decisions that now seem reasonable may later prove to be wrong. The best way to avoid problems is thus to keep one's options open.

Flexibility, along with transparency, is like parenthood and apple pie: who could possibly be against them? It is hard to imagine anyone arguing in favor of an inflexible system. However, things are not as simple as they seem. There are two schools of thought concerning the structure of distributed systems. One school maintains that each machine should run a traditional kernel that provides most services itself. The other maintains that the kernel should provide as little as possible, with the bulk of the operating system services available from user-level servers. These two models, known as the monolithic kernel and microkernel, respectively, are illustrated in Fig. 1-14.

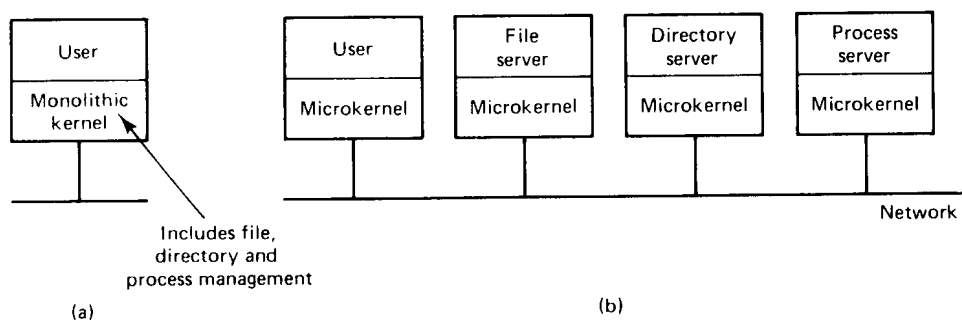


Fig. 1-14. (a) Monolithic kernel. (b) Microkernel.

The monolithic kernel is basically today's centralized operating system augmented with networking facilities and the integration of remote services. Most

system calls are made by trapping to the kernel, having the work performed there, and having the kernel return the desired result to the user process. With this approach, most machines have disks and manage their own local file systems. Many distributed systems that are extensions or imitations of UNIX use this approach because UNIX itself has a large, monolithic kernel.

If the monolithic kernel is the reigning champion, the microkernel is the up-and-coming challenger. Most distributed systems that have been designed from scratch use this method. The microkernel is more flexible because it does almost nothing. It basically provides just four minimal services:

1. An interprocess communication mechanism.
2. Some memory management.
3. A small amount of low-level process management and scheduling.
4. Low-level input/output.

In particular, unlike the monolithic kernel, it does not provide the file system, directory system, full process management, or much system call handling. The services that the microkernel does provide are included because they are difficult or expensive to provide anywhere else. The goal is to keep it small.

All the other operating system services are generally implemented as user-level servers. To look up a name, read a file, or obtain some other service, the user sends a message to the appropriate server, which then does the work and returns the result. The advantage of this method is that it is highly modular: there is a well-defined interface to each service (the set of messages the server understands), and every service is equally accessible to every client, independent of location. In addition, it is easy to implement, install, and debug new services, since adding or changing a service does not require stopping the system and booting a new kernel, as is the case with a monolithic kernel. It is precisely this ability to add, delete, and modify services that gives the microkernel its flexibility. Furthermore, users who are not satisfied with any of the official services are free to write their own.

As a simple example of this power, it is possible to have a distributed system with multiple file servers, one supporting MS-DOS file service and another supporting UNIX file service. Individual programs can use either or both, if they choose. In contrast, with a monolithic kernel, the file system is built into the kernel, and users have no choice but to use it.

The only potential advantage of the monolithic kernel is performance. Trapping to the kernel and doing everything there may well be faster than sending messages to remote servers. However, a detailed comparison of two distributed operating systems, one with a monolithic kernel (Sprite), and one with a microkernel (Amoeba), has shown that in practice this advantage is nonexistent

(Douglass et al., 1991). Other factors tend to dominate, and the small amount of time required to send a message and get a reply (typically, about 1 msec) is usually negligible. As a consequence, it is likely that microkernel systems will gradually come to dominate the distributed systems scheme, and monolithic kernels will eventually vanish or evolve into microkernels. Perhaps future editions of Silberschatz and Galvin's book on operating systems (1994) will feature hummingbirds and swifts on the cover instead of stegosauruses and triceratopses.

1.5.3. Reliability

One of the original goals of building distributed systems was to make them more reliable than single-processor systems. The idea is that if a machine goes down, some other machine takes over the job. In other words, theoretically the overall system reliability could be the Boolean OR of the component reliabilities. For example, with four file servers, each with a 0.95 chance of being up at any instant, the probability of all four being down simultaneously is $0.05^4 = 0.000006$, so the probability of at least one being available is 0.999994, far better than that of any individual server.

That is the theory. The practice is that to function at all, current distributed systems count on a number of specific servers being up. As a result, some of them have an availability more closely related to the Boolean AND of the components than to the Boolean OR. In a widely-quoted remark, Leslie Lamport once defined a distributed system as "one on which I cannot get any work done because some machine I have never heard of has crashed." While this remark was (presumably) made somewhat tongue-in-cheek, there is clearly room for improvement here.

It is important to distinguish various aspects of reliability. **Availability**, as we have just seen, refers to the fraction of time that the system is usable. Lamport's system apparently did not score well in that regard. Availability can be enhanced by a design that does not require the simultaneous functioning of a substantial number of critical components. Another tool for improving availability is redundancy: key pieces of hardware and software should be replicated, so that if one of them fails the others will be able to take up the slack.

A highly reliable system must be highly available, but that is not enough. Data entrusted to the system must not be lost or garbled in any way, and if files are stored redundantly on multiple servers, all the copies must be kept consistent. In general, the more copies that are kept, the better the availability, but the greater the chance that they will be inconsistent, especially if updates are frequent. The designers of all distributed systems must keep this dilemma in mind all the time.

Another aspect of overall reliability is security. Files and other resources must be protected from unauthorized usage. Although the same issue occurs in

single-processor systems, in distributed systems it is more severe. In a single-processor system, the user logs in and is authenticated. From then on, the system knows who the user is and can check whether each attempted access is legal. In a distributed system, when a message comes in to a server asking for something, the server has no simple way of determining who it is from. No name or identification field in the message can be trusted, since the sender may be lying. At the very least, considerable care is required here.

Still another issue relating to reliability is **fault tolerance**. Suppose that a server crashes and then quickly reboots. What happens? Does the server crash bring users down with it? If the server has tables containing important information about ongoing activities, recovery will be difficult at best.

In general, distributed systems can be designed to mask failures, that is, to hide them from the users. If a file service or other service is actually constructed from a group of closely cooperating servers, it should be possible to construct it in such a way that users do not notice the loss of one or two servers, other than some performance degradation. Of course, the trick is to arrange this cooperation so that it does not add substantial overhead to the system in the normal case, when everything is functioning correctly.

1.5.4. Performance

Always lurking in the background is the issue of performance. Building a transparent, flexible, reliable distributed system will not win you any prizes if it is as slow as molasses. In particular, when running a particular application on a distributed system, it should not be appreciably worse than running the same application on a single processor. Unfortunately, achieving this is easier said than done.

Various performance metrics can be used. Response time is one, but so are throughput (number of jobs per hour), system utilization, and amount of network capacity consumed. Furthermore, the results of any benchmark are often highly dependent on the nature of the benchmark. A benchmark that involves a large number of independent highly CPU-bound computations may give radically different results from a benchmark that consists of scanning a single large file for some pattern.

The performance problem is compounded by the fact that communication, which is essential in a distributed system (and absent in a single-processor system) is typically quite slow. Sending a message and getting a reply over a LAN takes about 1 msec. Most of this time is due to unavoidable protocol handling on both ends, rather than the time the bits spend on the wire. Thus to optimize performance, one often has to minimize the number of messages. The difficulty with this strategy is that the best way to gain performance is to have many activities running in parallel on different processors, but doing so requires

sending many messages. (Another solution is to do all the work on one machine, but that is hardly appropriate in a distributed system.)

One possible way out is to pay considerable attention to the **grain size** of all computations. Starting up a small computation remotely, such as adding two integers, is rarely worth it, because the communication overhead dwarfs the extra CPU cycles gained. On the other hand, starting up a long compute-bound job remotely may be worth the trouble. In general, jobs that involve a large number of small computations, especially ones that interact highly with one another, may cause trouble on a distributed system with relatively slow communication. Such jobs are said to exhibit **fine-grained parallelism**. On the other hand, jobs that involve large computations, low interaction rates, and little data, that is, **coarse-grained parallelism**, may be a better fit.

Fault tolerance also exacts its price. Good reliability is often best achieved by having several servers closely cooperating on a single request. For example, when a request comes in to a server, it could immediately send a copy of the message to one of its colleagues so that if it crashes before finishing, the colleague can take over. Naturally, when it is done, it must inform the colleague that the work has been completed, which takes another message. Thus we have at least two extra messages, which in the normal case cost time and network capacity and produce no tangible gain.

1.5.5. Scalability

Most current distributed systems are designed to work with a few hundred CPUs. It is possible that future systems will be orders of magnitude larger, and solutions that work well for 200 machines will fail miserably for 200,000,000. Consider the following. The French PTT (Post, Telephone and Telegraph administration) is in the process of installing a terminal in every household and business in France. The terminal, known as a **minitel**, will allow online access to a data base containing all the telephone numbers in France, thus eliminating the need for printing and distributing expensive telephone books. It will also vastly reduce the need for information operators who do nothing but give out telephone numbers all day. It has been calculated that the system will pay for itself within a few years. If the system works in France, other countries will inevitably adopt similar systems.

Once all the terminals are in place, the possibility of also using them for electronic mail (especially in conjunction with printers) is clearly present. Since postal services lose a huge amount of money in every country in the world, and telephone services are enormously profitable, there are great incentives to having electronic mail replace paper mail.

Next comes interactive access to all kinds of data bases and services, from

electronic banking to reserving places in planes, trains, hotels, theaters, and restaurants, to name just a few. Before long, we have a distributed system with tens of millions of users. The question is: Will the methods we are currently developing scale to such large systems?

Although little is known about such huge distributed systems, one guiding principle is clear: avoid centralized components, tables, and algorithms (see Fig. 1-15). Having a single mail server for 50 million users would not be a good idea. Even if it had enough CPU and storage capacity, the network capacity into and out of it would surely be a problem. Furthermore, the system would not tolerate faults well. A single power outage could bring the entire system down. Finally, most mail is local. Having a message sent by a user in Marseille to another user two blocks away pass through a machine in Paris is not the way to go.

Concept	Example
Centralized components	A single mail server for all users
Centralized tables	A single on-line telephone book
Centralized algorithms	Doing routing based on complete information

Fig. 1-15. Potential bottlenecks that designers should try to avoid in very large distributed systems.

Centralized tables are almost as bad as centralized components. How should one keep track of the telephone numbers and addresses of 50 million people? Suppose that each data record could be fit into 50 characters. A single 2.5-gigabyte disk would provide enough storage. But here again, having a single data base would undoubtedly saturate all the communication lines into and out of it. It would also be vulnerable to failures (a single speck of dust could cause a head crash and bring down the entire directory service). Furthermore, here too, valuable network capacity would be wasted shipping queries far away for processing.

Finally, centralized algorithms are also a bad idea. In a large distributed system, an enormous number of messages have to be routed over many lines. From a theoretical point of view, the optimal way to do this is collect complete information about the load on all machines and lines, and then run a graph theory algorithm to compute all the optimal routes. This information can then be spread around the system to improve the routing.

The trouble is that collecting and transporting all the input and output information would again be a bad idea for the reasons discussed above. In fact, any algorithm that operates by collecting information from all sites, sends it to a single machine for processing, and then distributes the results must be avoided.

Only decentralized algorithms should be used. These algorithms generally have the following characteristics, which distinguish them from centralized algorithms:

1. No machine has complete information about the system state.
2. Machines make decisions based only on local information.
3. Failure of one machine does not ruin the algorithm.
4. There is no implicit assumption that a global clock exists.

The first three follow from what we have said so far. The last is perhaps less obvious, but also important. Any algorithm that starts out with: “At precisely 12:00:00 all machines shall note the size of their output queue” will fail because it is impossible to get all the clocks exactly synchronized. Algorithms should take into account the lack of exact clock synchronization. The larger the system, the larger the uncertainty. On a single LAN, with considerable effort it may be possible to get all clocks synchronized down to a few milliseconds, but doing this nationally is tricky. We will discuss distributed clock synchronization in Chap. 3.

1.6. SUMMARY

Distributed systems consist of autonomous CPUs that work together to make the complete system look like a single computer. They have a number of potential selling points, including good price/performance ratios, the ability to match distributed applications well, potentially high reliability, and incremental growth as the workload grows. They also have some disadvantages, such as more complex software, potential communication bottlenecks, and weak security. Nevertheless, there is considerable interest worldwide in building and installing them.

Modern computer systems often have multiple CPUs. These can be organized as multiprocessors (with shared memory) or as multicomputers (without shared memory). Both types can be bus-based or switched. The former tend to be tightly coupled, while the latter tend to be loosely coupled.

The software for multiple CPU systems can be divided into three rough classes. Network operating systems allow users at independent workstations to communicate via a shared file system but otherwise leave each user as the master of his own workstation. Distributed operating systems turn the entire collection of hardware and software into a single integrated system, much like a traditional timesharing system. Shared-memory multiprocessors also offer a single

system image, but do so by centralizing everything, so there really is only a single system. Shared-memory multiprocessors are not distributed systems.

Distributed systems have to be designed carefully, since there are many pitfalls for the unwary. A key issue is transparency—hiding all the distribution from the users and even from the application programs. Another issue is flexibility. Since the field is only now in its infancy, the design should be made with the idea of making future changes easy. In this respect, microkernels are superior to monolithic kernels. Other important issues are reliability, performance, and scalability.

PROBLEMS

1. The price/performance ratio of computers has improved by something like 11 orders of magnitude since the first commercial mainframes came out in the early 1950s. The text shows what a similar gain would have meant in the automobile industry. Give another example of what such a large gain means.
2. Name two advantages and two disadvantages of distributed systems over centralized ones.
3. What is the difference between a multiprocessor and a multicomputer?
4. The terms *loosely-coupled system* and *tightly-coupled system* are often used to describe distributed computer systems. What is the difference between them?
5. What is the difference between an MIMD computer and an SIMD computer?
6. A bus-based multiprocessor uses snoopy caches to achieve a coherent memory. Will semaphores work on this machine?
7. Crossbar switches allow a large number of memory requests to be processed at once, giving excellent performance. Why are they rarely used in practice?
8. A multicomputer with 256 CPUs is organized as a 16×16 grid. What is the worst-case delay (in hops) that a message might have to take?
9. Now consider a 256-CPU hypercube. What is the worst-case delay here, again in hops?
10. A multiprocessor has 4096 50-MIPS CPUs connected to memory by an omega network. How fast do the switches have to be to allow a request to

go to memory and back in one instruction time?

11. What is meant by a *single-system image*?
12. What is the main difference between a distributed operating system and a network operating system?
13. What are the primary tasks of a microkernel?
14. Name two advantages of a microkernel over a monolithic kernel.
15. Concurrency transparency is a desirable goal for distributed systems. Do centralized systems have this property automatically?
16. Explain in your own words the concept of parallelism transparency.
17. An experimental file server is up 3/4 of the time and down 1/4 of the time, due to bugs. How many times does this file server have to be replicated to give an availability of at least 99 percent?
18. Suppose that you have a large source program consisting of m files to compile. The compilation is to take place on a system with n processors, where $n \gg m$. The best you can hope for is an m -fold speedup over a single processor. What factors might cause the speedup to be less than this maximum?

2

Communication in Distributed Systems

The single most important difference between a distributed system and a uniprocessor system is the interprocess communication. In a uniprocessor system, most interprocess communication implicitly assumes the existence of shared memory. A typical example is the producer-consumer problem, in which one process writes into a shared buffer and another process reads from it. Even that most basic form of synchronization, the semaphore, requires that one word (the semaphore variable itself) is shared. In a distributed system there is no shared memory whatsoever, so the entire nature of interprocess communication must be completely rethought from scratch. In this chapter we will discuss numerous issues, examples, and problems associated with interprocess communication in distributed operating systems.

We will start out by discussing the rules that communicating processes must adhere to, known as protocols. For wide-area distributed systems these protocols often take the form of multiple layers, each with its own goals and rules. Two sets of layers, OSI and ATM, will be examined. Then we will look at the client-server model in some detail. After that, it is time to find out how messages are exchanged and the many options available to system designers.

One particular option, remote procedure call, is important enough to warrant its own section. Remote procedure call is really a nicer way of packaging message passing, to make it more like conventional programming and easier to use. Nevertheless, it has its own peculiarities and issues, which we will also look at.

We will conclude the chapter by studying how groups of processes can communicate, instead of just two processes. A detailed example of group communication, ISIS, will be discussed.

2.1. LAYERED PROTOCOLS

Due to the absence of shared memory, all communication in distributed systems is based on message passing. When process *A* wants to communicate with process *B*, it first builds a message in its own address space. Then it executes a system call that causes the operating system to fetch the message and send it over the network to *B*. Although this basic idea sounds simple enough, in order to prevent chaos, *A* and *B* have to agree on the meaning of the bits being sent. If *A* sends a brilliant new novel written in French and encoded in IBM's EBCDIC character code, and *B* expects the inventory of a supermarket written in English and encoded in ASCII, communication will be less than optimal.

Many different agreements are needed. How many volts should be used to signal a 0-bit, and how many volts for a 1-bit? How does the receiver know which is the last bit of the message? How can it detect if a message has been damaged or lost, and what should it do if it finds out? How long are numbers, strings, and other data items, and how are they represented? In short, agreements are needed at a variety of levels, varying from the low-level details of bit transmission to the high-level details of how information is to be expressed.

To make it easier to deal with the numerous levels and issues involved in communication, the International Standards Organization (ISO) has developed a reference model that clearly identifies the various levels involved, gives them standard names, and points out which level should do which job. This model is called the **Open Systems Interconnection Reference Model** (Day and Zimmerman, 1983), usually abbreviated as **ISO OSI** or sometimes just the **OSI model**. Although we do not intend to give a full description of this model and all of its implications here, a short introduction will be helpful. For more details, see (Tanenbaum, 1988).

To start with, the OSI model is designed to allow open systems to communicate. An **open system** is one that is prepared to communicate with any other open system by using standard rules that govern the format, contents, and meaning of the messages sent and received. These rules are formalized in what are called **protocols**. Basically, a protocol is an agreement between the communicating parties on how communication is to proceed. When a woman is introduced to a man, she may choose to stick out her hand. He, in turn, may decide either to shake it or kiss it, depending, for example, whether she is an American lawyer at a business meeting or a European princess at a formal ball. Violating the protocol will make communication more difficult, if not impossible.

At a more technological level, many companies make memory boards for the IBM PC. When the CPU wants to read a word from memory, it puts the address and certain control signals on the bus. The memory board is expected to see these signals and respond by putting the word requested on the bus within a certain time interval. If the memory board observes the required bus protocol, it will work correctly, otherwise it will not.

Similarly, to allow a group of computers to communicate over a network, they must all agree on the protocols to be used. The OSI model distinguishes between two general types of protocols. With **connection-oriented** protocols, before exchanging data, the sender and receiver first explicitly establish a connection, and possibly negotiate the protocol they will use. When they are done, they must release (terminate) the connection. The telephone is a connection-oriented communication system. With **connectionless** protocols, no setup in advance is needed. The sender just transmits the first message when it is ready. Dropping a letter in a mailbox is an example of connectionless communication. With computers, both connection-oriented and connectionless communication are common.

In the OSI model, communication is divided up into seven levels or layers, as shown in Fig. 2-1. Each layer deals with one specific aspect of the communication. In this way, the problem can be divided up into manageable pieces, each of which can be solved independent of the others. Each layer provides an **interface** to the one above it. The interface consists of a set of operations that together define the service the layer is prepared to offer its users.

In the OSI model, when process *A* on machine 1 wants to communicate with process *B* on machine 2, it builds a message and passes the message to the application layer on its machine. This layer might be a library procedure, for example, but it could also be implemented in some other way (e.g., inside the operating system, on an external coprocessor chip, etc.). The application layer software then adds a **header** to the front of the message and passes the resulting message across the layer 6/7 interface to the presentation layer. The presentation layer in turn adds its own header and passes the result down to the session layer, and so on. Some layers add not only a header to the front, but also a trailer to the end. When it hits bottom, the physical layer actually transmits the message, which by now might look as shown in Fig. 2-2.

When the message arrives at machine 2, it is passed upward, with each layer stripping off and examining its own header. Finally, the message arrives at the receiver, process *B*, which may reply to it using the reverse path. The information in the layer *n* header is used for the layer *n* protocol.

As an example of why layered protocols are important, consider communication between two companies, Zippy Airlines and its caterer, Mushy Meals, Inc. Every month, the head of passenger service at Zippy asks her secretary to contact the sales manager's secretary at Mushy to order 100,000 boxes of rubber

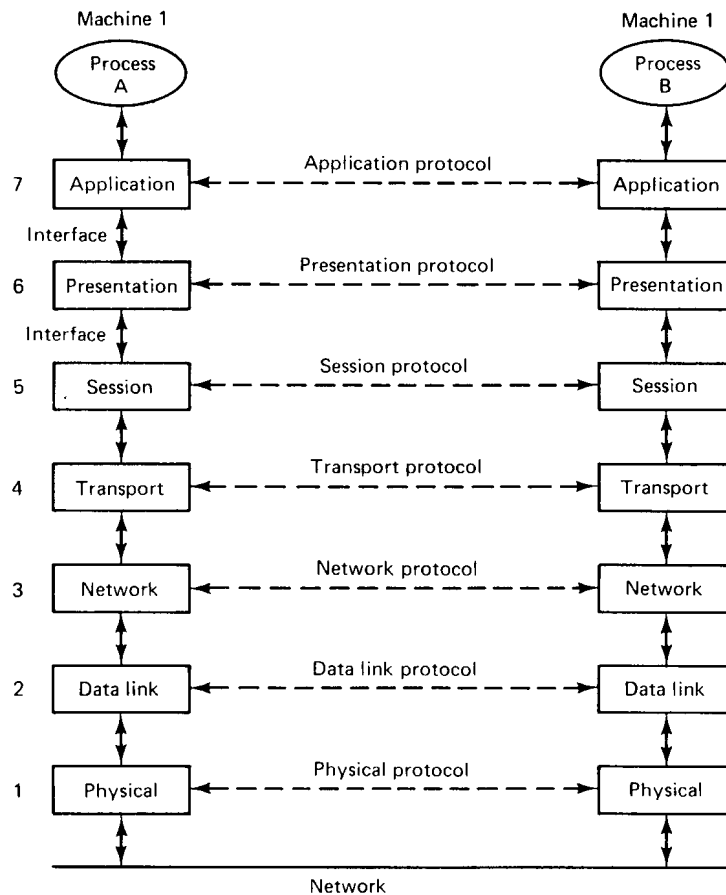


Fig. 2-1. Layers, interfaces, and protocols in the OSI model.

chicken. Traditionally, the orders have gone via the post office. However, as the postal service deteriorates, at some point the two secretaries decide to abandon it and communicate by FAX. They can do this without bothering their bosses, since their protocol deals with the physical transmission of the orders, not their contents.

Similarly, the head of passenger service can decide to drop the rubber chicken and go for Mushy's new special, prime rib of goat, without that decision affecting the secretaries. The thing to notice is that we have two layers here, the bosses and the secretaries. Each layer has its own protocol (subjects of discussion and technology) that can be changed independently of the other one. It is precisely this independence that makes layered protocols attractive. Each one can be changed as technology improves, without the other ones being affected.

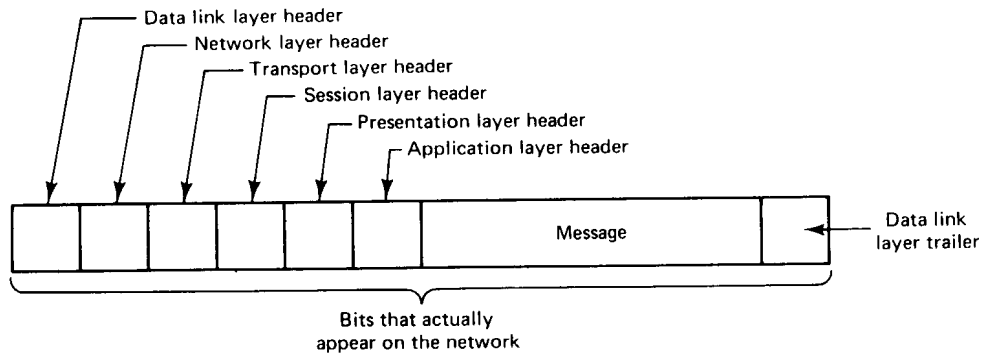


Fig. 2-2. A typical message as it appears on the network.

In the OSI model, there are not two layers, but seven, as we saw in Fig. 2-1. The collection of protocols used in a particular system is called a **protocol suite** or **protocol stack**. In the following sections, we will briefly examine each of the layers in turn, starting at the bottom. Where appropriate, we will also point out some of the protocols used in each layer.

2.1.1. The Physical Layer

The physical layer is concerned with transmitting the 0s and 1s. How many volts to use for 0 and 1, how many bits per second can be sent, and whether transmission can take place in both directions simultaneously are key issues in the physical layer. In addition, the size and shape of the network connector (plug), as well as the number of pins and meaning of each are of concern here.

The physical layer protocol deals with standardizing the electrical, mechanical, and signaling interfaces so that when one machine sends a 0 bit it is actually received as a 0 bit and not a 1 bit. Many physical layer standards have been developed (for different media), for example, the RS-232-C standard for serial communication lines.

2.1.2. The Data Link Layer

The physical layer just sends bits. As long as no errors occur, all is well. However, real communication networks are subject to errors, so some mechanism is needed to detect and correct them. This mechanism is the main task of the data link layer. What it does is to group the bits into units, sometimes called **frames**, and see that each frame is correctly received.

The data link layer does its work by putting a special bit pattern on the start and end of each frame, to mark them, as well as computing a **checksum** by

adding up all the bytes in the frame in a certain way. The data link layer appends the checksum to the frame. When the frame arrives, the receiver recomputes the checksum from the data and compares the result to the checksum following the frame. If they agree, the frame is considered correct and is accepted. If they disagree, the receiver asks the sender to retransmit it. Frames are assigned sequence numbers (in the header), so everyone can tell which is which.

In Fig. 2-3 we see a (slightly pathological) example of *A* trying to send two messages, 0 and 1, to *B*. At time 0, data message 0 is sent, but when it arrives, at time 1, noise on the transmission line has caused it to be damaged, so the checksum is wrong. *B* notices this, and at time 2 asks for a retransmission using a control message. Unfortunately, at the same time, *A* is sending data message 1. When *A* gets the request for retransmission, it resends 0. However, when *B* gets message 1, instead of the requested message 0, it sends control message 1 to *A* complaining that it wants 0, not 1. When *A* sees this, it shrugs its shoulders and sends message 0 for the third time.

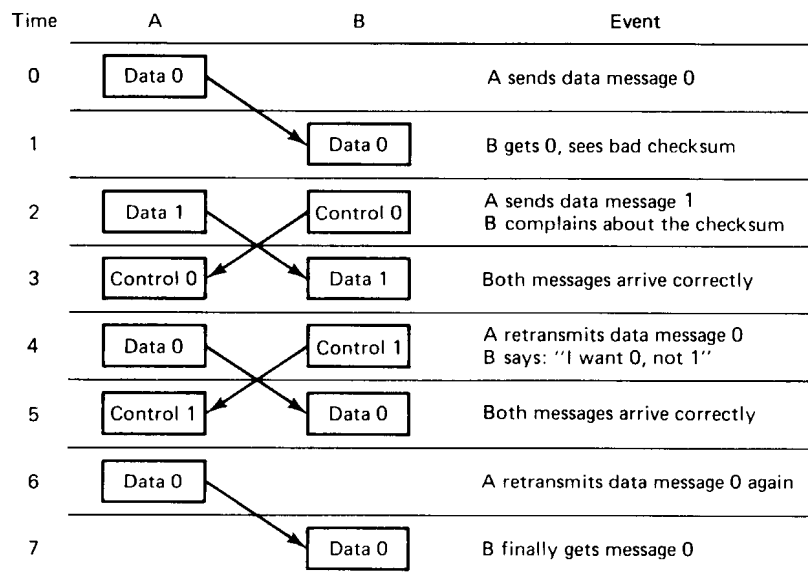


Fig. 2-3. Discussion between a receiver and a sender in the data link layer.

The point here is not so much whether the protocol of Fig. 2-3 is a great one (it is not), but rather to illustrate that in each layer there is a need for discussion between the sender and the receiver. Typical messages are "Please retransmit message *n*," "I already retransmitted it," "No you did not," "Yes I did," "All right, have it your way, but send it again," and so forth. This discussion takes

place in the header field, where various requests and responses are defined, and parameters (such as frame numbers) can be supplied.

2.1.3. The Network Layer

On a LAN, there is usually no need for the sender to locate the receiver. It just puts the message out on the network and the receiver takes it off. A wide-area network, however, consists of a large number of machines, each with some number of lines to other machines, rather like a large-scale map showing major cities and roads connecting them. For a message to get from the sender to the receiver it may have to make a number of hops, at each one choosing an outgoing line to use. The question of how to choose the best path is called **routing**, and is the primary task of the network layer.

The problem is complicated by the fact that the shortest route is not always the best route. What really matters is the amount of delay on a given route, which, in turn, is related to the amount of traffic and the number of messages queued up for transmission over the various lines. The delay can thus change over the course of time. Some routing algorithms try to adapt to changing loads, whereas others are content to make decisions based on long-term averages.

Two network-layer protocols are in widespread use, one connection-oriented and one connectionless. The connection-oriented one is called **X.25**, and is favored by the operators of public networks, such as telephone companies and the European PTTs. The X.25 user first sends a *Call Request* to the destination, which can either accept or reject the proposed connection. If the connection is accepted, the caller is given a connection identifier to use in subsequent requests. In many cases, the network chooses a route from the sender to the receiver during this setup, and uses it for subsequent traffic.

The connectionless one is called **IP** (Internet Protocol) and is part of the DoD (U.S. Department of Defense) protocol suite. An **IP packet** (the technical term for a message in the network layer) can be sent without any setup. Each IP packet is routed to its destination independent of all others. No internal path is selected and remembered as is often the case with X.25.

2.1.4. The Transport Layer

Packets can be lost on the way from the sender to the receiver. Although some applications can handle their own error recovery, others prefer a reliable connection. The job of the transport layer is to provide this service. The idea is that the session layer should be able to deliver a message to the transport layer with the expectation that it will be delivered without loss.

Upon receiving a message from the session layer, the transport layer breaks it into pieces small enough for each to fit in a single packet, assigns each one a

sequence number, and then sends them all. The discussion in the transport layer header concerns which packets have been sent, which have been received, how many more the receiver has room to accept, and similar topics.

Reliable transport connections (which by definition are connection-oriented) can be built on top of either X.25 or IP. In the former case all the packets will arrive in the correct sequence (if they arrive at all), but in the latter case it is possible for one packet to take a different route and arrive earlier than the packet sent before it. It is up to the transport layer software to put everything back in order to maintain the illusion that a transport connection is like a big tube—you put messages into it and they come out undamaged and in the same order in which they went in.

The official ISO transport protocol has five variants, known as **TP0** through **TP4**. The differences relate to error handling and the ability to send several transport connections over a single X.25 connection. The choice of which one to use depends on the properties of the underlying network layer.

The DoD transport protocol is called **TCP (Transmission Control Protocol)** and is described in detail in (Comer, 1991). It is similar to TP4. The combination TCP/IP is widely used at universities and on most UNIX systems. The DoD protocol suite also supports a connectionless transport protocol called **UDP (Universal Datagram Protocol)**, which is essentially just IP with some minor additions. User programs that do not need a connection-oriented protocol normally use UDP.

2.1.5. The Session Layer

The session layer is essentially an enhanced version of the transport layer. It provides dialog control, to keep track of which party is currently talking, and it provides synchronization facilities. The latter are useful to allow users to insert checkpoints into long transfers, so that in the event of a crash it is only necessary to go back to the last checkpoint, rather than all the way back to the beginning. In practice, few applications are interested in the session layer and it is rarely supported. It is not even present in the DoD protocol suite.

2.1.6. The Presentation Layer

Unlike the lower layers, which are concerned with getting the bits from the sender to the receiver reliably and efficiently, the presentation layer is concerned with the meaning of the bits. Most messages do not consist of random bit strings, but more structured information such as people's names, addresses, amounts of money, and so on. In the presentation layer it is possible to define

records containing fields like these and then have the sender notify the receiver that a message contains a particular record in a certain format. This makes it easier for machines with different internal representations to communicate.

2.1.7. The Application Layer

The application layer is really just a collection of miscellaneous protocols for common activities such as electronic mail, file transfer, and connecting remote terminals to computers over a network. The best known of these are the X.400 electronic mail protocol and the X.500 directory server. Neither this layer nor the two layers directly under it will be of interest to us in this book.

2.2. ASYNCHRONOUS TRANSFER MODE NETWORKS

The OSI world sketched in the previous section was developed in the 1970s and implemented (to some extent) in the 1980s. New developments in the 1990s are overtaking OSI, certainly in the technology-driven lower layers. In this section we will touch just briefly on some of these advances in networking, since future distributed systems will very likely be built on them, and it is important for operating system designers to be aware of them. For a more complete treatment of the state-of-the-art in network technology, see (Kleinrock, 1992; and Partridge, 1993, 1994).

In the past quarter century, computers have improved in performance by *many* orders of magnitude. Networks have not. When the ARPANET, the predecessor to the Internet, was inaugurated in 1969, it used 56 Kbps communication lines between the nodes. This was state-of-the-art communication then. In the late 1970s and early 1980s, many of these lines were replaced by T1 lines running at 1.5 Mbps. Eventually, the main backbone evolved into a T3 network at 45 Mbps, but most lines on the Internet are still T1 or slower.

New developments are suddenly about to make 155 Mbps the low-end standard, with major trunks running at 1 gigabit/sec and up (Catlett, 1992; Cheung, 1992; and Lyles and Swinehart, 1992). This rapid change will have an enormous impact on distributed systems, making possible all kinds of applications that were previously unthinkable, but it also brings new challenges. It is this new technology that we will describe below.

2.2.1. What Is Asynchronous Transfer Mode?

In the late 1980s, the world's telephone companies finally began to realize that there was more to telecommunications than transmitting voice in 4 KHz analog channels. It is true that data networks, such as X.25, existed for years,

but they were clearly stepchildren and frequently ran at 56 or 64 Kbps. Systems like the Internet were regarded as academic curiosities, akin to a two-headed cow in a circus sideshow. Analog voice was where the action (and money) was.

When the telephone companies decided to build networks for the 21st Century, they faced a dilemma: voice traffic is smooth, needing a low, but constant bandwidth, whereas data traffic is bursty, usually needing no bandwidth (when there is no traffic), but sometimes needing a great deal for very short periods of time. Neither traditional circuit switching (used in the Public Switched Telephone Network) nor packet switching (used in the Internet) was suitable for both kinds of traffic.

After much study, a hybrid form using fixed-size blocks over virtual circuits was chosen as a compromise that gave reasonably good performance for both types of traffic. This scheme, called **ATM (Asynchronous Transfer Mode)** has become an international standard and is likely to play a major role in future distributed systems, both local-area ones and wide-area ones. For tutorials on ATM, see (Le Boudec, 1992; Minzer, 1989; and Newman, 1994).

The ATM model is that a sender first establishes a connection (i.e., a virtual circuit) to the receiver or receivers. During connection establishment, a route is determined from the sender to the receiver(s) and routing information is stored in the switches along the way. Using this connection, packets can be sent, but they are chopped up by the hardware into small, fixed-sized units called **cells**. The cells for a given virtual circuit all follow the path stored in the switches. When the connection is no longer needed, it is released and the routing information purged from the switches.

This scheme has a number of advantages over traditional packet and circuit switching. The most important one is that a single network can now be used to transport an arbitrary mix of voice, data, broadcast television, videotapes, radio, and other information efficiently, replacing what were previously separate networks (telephone, X.25, cable TV, etc.). New services, such as video conferencing for businesses, will also use it. In all cases, what the network sees is cells; it does not care what is in them. This integration represents an enormous cost saving and simplification that will make it possible for each home and business to have a single wire (or fiber) coming in for all its communication and information needs. It will also make possible new applications, such as video-on-demand, teleconferencing, and access to thousands of remote data bases.

Cell switching lends itself well to multicasting (one cell going to many destinations), a technique needed for transmitting broadcast television to thousands of houses at the same time. Conventional circuit switching, as used in the telephone system, cannot handle this. Broadcast media, such as cable TV can, but they cannot handle point-to-point traffic without wasting bandwidth (effectively broadcasting every message). The advantage of cell switching is that it can handle both point-to-point and multicasting efficiently.

Fixed-size cells allow rapid switching, something much harder to achieve with current store-and-forward packet switches. They also eliminate the danger of a small packet being delayed because a big one is hogging a needed line. With cell switching, after each cell is transmitted, a new one can be sent, even a new one belonging to a different packet.

ATM has its own protocol hierarchy, as shown in Fig. 2-4. The physical layer has the same functionality as layer 1 in the OSI model. The ATM layer deals with cells and cell transport, including routing, so it covers OSI layer 2 and part of layer 3. However, unlike OSI layer 2, the ATM layer does not recover lost or damaged cells. The adaptation layer handles breaking packets into cells and reassembling them at the other end, which does not appear explicitly in the OSI model until layer 4. The service offered by the adaptation layer is not a perfectly reliable end-to-end service, so transport connections must be implemented in the upper layers, for example, by using ATM cells to carry TCP/IP traffic.

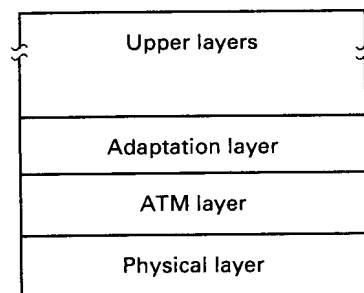


Fig. 2-4. The ATM reference model.

In the following sections, we will examine the lowest three layers of Fig. 2-4 in turn, starting at the bottom and working our way up.

2.2.2. The ATM Physical Layer

An ATM adaptor board plugged into a computer can put out a stream of cells onto a wire or fiber. The transmission stream must be continuous. When there are no data to be sent, empty cells are transmitted, which means that in the physical layer, ATM is really synchronous, not asynchronous. Within a virtual circuit, however, it is asynchronous.

Alternatively, the adaptor board can use **SONET (Synchronous Optical Network)** in the physical layer, putting its cells into the payload portion of SONET frames. The virtue of this approach is compatibility with the internal transmission system of AT&T and other carriers that use SONET. In Europe, a system called **SDH (Synchronous Digital Hierarchy)** that is closely patterned after SONET is available in some countries.

In SONET, the basic unit (analogous to a 193-bit T1 frame) is a 9×90 array of bytes called a **frame**. Of these 810 bytes, 36 bytes are overhead, leaving 774 bytes of payload. One frame is transmitted every 125 μ sec, to match the telephone system's standard sampling rate of 8000 samples/sec, so the gross data rate (including overhead) is 51.840 Mbps and the net data rate (excluding overhead) is 49.536 Mbps.

These parameters were chosen after five years of tortuous negotiation between U.S., European, Japanese, and other telephone companies in order to handle the U.S. T3 data stream (44.736 Mbps) and the standards used by other countries. The computer industry did not play a significant role here (a 9×90 array with 36 bytes of overhead is not something a computer scientist is likely to propose).

The basic 51.840-Mbps channel is called **OC-1**. It is possible to send a group of n OC-1 frames as a group, which is designated OC- n when it is used for n independent OC-1 channels and OC- nc (for concatenated) when used for a single high-speed channel. Standards have been established for OC-3, OC-12, OC-48, and OC-192. The most important of these for ATM are OC-3c, at 155.520 Mbps and OC-12c, at 622.080 Mbps, because computers can probably produce data at these rates in the near future. For long-haul transmission within the telephone system, OC-12 and OC-48 are the most widely used at present.

OC-3c SONET adaptors for computers are now available to allow a computer to output SONET frames directly. OC-12c is expected shortly. Since even OC-1 is overkill for a telephone, it is unlikely that many audio telephones will ever speak ATM or SONET directly (ISDN will be used instead), but for videophones ATM and SONET are ideal.

2.2.3. The ATM Layer

When ATM was being developed, two factions developed within the standards committee. The Europeans wanted 32-byte cells because these had a small enough delay that echo suppressors would not be needed in most European countries. The Americans, who already had echo suppressors, wanted 64-byte cells due to their greater efficiency for data traffic.

The end result was a 48-byte cell, which no one really liked. It is too big for voice and too small for data. To make it even worse, a 5-byte header was added, giving a 53-byte cell containing a 48-byte data field. Note that a 53-byte cell is not a good match for a 774-byte SONET payload, so ATM cells will span SONET frames. Two separate levels of synchronization are thus needed: one to detect the start of a SONET frame, and one to detect the start of the first full ATM cell within the SONET payload. However, a standard for packing ATM cells into SONET frames exists, and the entire layer can be done in hardware.

The layout of a cell header from a computer to the first ATM switch is shown in Fig. 2-5. Unfortunately, the layout of a cell header between two ATM switches is different, with the *GFC* field being replaced by four more bits for the *VPI* field. In the view of many, this is unfortunate, since it introduces an unnecessary distinction between computer-to-switch and switch-to-switch cells and hence adaptor hardware. Both kinds of cells have 48-byte payloads directly following the header.

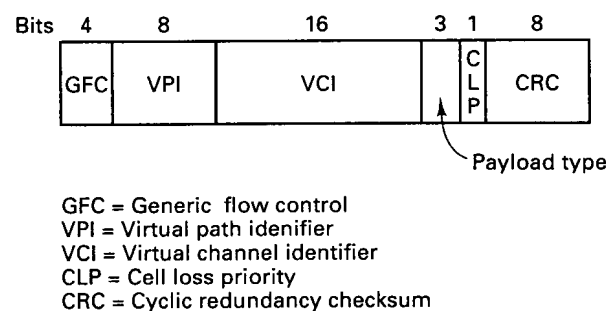


Fig. 2-5. User-to-network cell header layout.

The *GFC* may some day be used for flow control, if an agreement on how to do it can be achieved. The *VPI* and *VCI* fields together identify which path and virtual circuit a cell belongs to. Routing tables along the way use this information for routing. These fields are modified at each hop along the path. The purpose of the *VPI* field is to group together a collection of virtual circuits for the same destination and make it possible for a carrier to reroute all of them without having to examine the *VCI* field.

The *Payload type* field distinguishes data cells from control cells, and further identifies several kinds of control cells. The *CLP* field can be used to mark some cells as less important than others, so if congestion occurs, the least important ones will be the ones dropped. Finally, there is a 1-byte checksum over the header (but not the data).

2.2.4. The ATM Adaptation Layer

At 155 Mbps, a cell can arrive every 3 μ sec. Few, if any, current CPUs can handle in excess of 300,000 interrupts/sec. Thus a mechanism is needed to allow a computer to send a packet and to have the ATM hardware break it into cells, transmit the cells, and then have them reassembled at the other end, generating one interrupt per packet, not per cell. This disassembly/reassembly is the job of the adaptation layer. It is expected that most host adaptor boards will

run the adaptation layer on the board and give one interrupt per incoming packet, not one per incoming cell.

Unfortunately, here too, the standards writers did not get it quite right. Originally adaptation layers were defined for four classes of traffic:

1. Constant bit rate traffic (for audio and video).
2. Variable bit rate traffic but with bounded delay.
3. Connection-oriented data traffic.
4. Connectionless data traffic.

Quickly it was discovered that classes 3 and 4 were essentially the same, so they were merged into a new class, 3/4. At that point the computer industry woke up from a short nap and noticed that none of the adaptation layers were suitable for data traffic, so they drafted AAL 5, for computer-to-computer traffic (Suzuki, 1994). Its nickname, **SEAL (Simple and Efficient Adaptation Layer)**, hints at what its designers thought of the other three AAL layers. (In all fairness, it should be pointed out that getting people from two industries with very different traditions, telephony and computers, to agree to a standard at all was a nontrivial achievement.)

Let us focus on SEAL, due to its simplicity. It uses only one bit in the ATM header, one of the bits in the *Payload type* field. This bit is normally 0, but is set to 1 in the last cell of a packet. The last cell contains a trailer in the final 8 bytes. In most cases there will be some padding (with zeros) between the end of the packet and the start of the trailer. With SEAL, the destination just assembles incoming cells for each virtual circuit until it finds one with the end-of-packet bit set. Then it extracts and processes the trailer.

The trailer has four fields. The first two are each 1 byte long and are not used. Then comes a 2-byte field giving the packet length, and a 4-byte checksum over the packet, padding, and trailer.

2.2.5. ATM Switching

ATM networks are built up of copper or optical cables and switches. Figure 2-6(a) illustrates a network with four switches. Cells originating at any of the eight computers attached to the system can be switched to any of the other computers by traversing one or more switches. Each of these switches has four ports, each used for both input and output.

The inside of a generic switch is illustrated in Fig. 2-6(b). It has input lines and output lines and a parallel **switching fabric** that connects them. Because a cell has to be switched in 3 μ sec (at OC-3), and as many cells as there are input lines can arrive at once, parallel switching is essential.

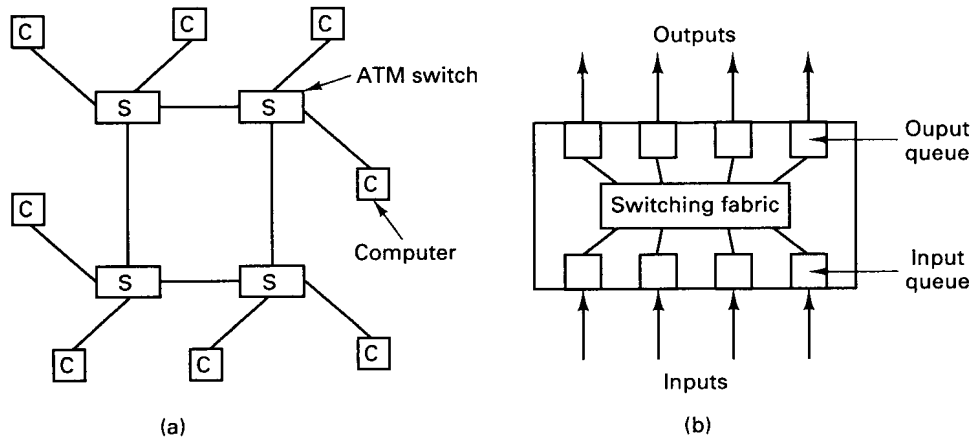


Fig. 2-6. (a) An ATM switching network. (b) Inside one switch.

When a cell arrives, its VPI and VCI fields are examined. Based on these and information stored in the switch when the virtual circuit was established, the cell is routed to the correct output port. Although the standard allows cells to be dropped, it requires that those delivered must be delivered in order.

A problem arises when two cells arrive at the same time on different input lines and need to go to the same output port. Just throwing one of them away is allowed by the standard, but if your switch drops more than 1 cell in 10^{12} , you are unlikely to sell many switches. An alternative scheme is to pick one of them at random and forward it, holding the other cell until later. In the next round, this algorithm is applied again. If two ports each have streams of cells for the same destination, substantial input queues will build up, blocking other cells behind them that want to go to output ports that are free. This problem is known as **head-of-line blocking**.

A different switch design copies the cell into a queue associated with the output buffer and lets it wait there, instead of keeping it in the input buffer. This approach eliminates head-of-line blocking and gives better performance. It is also possible for a switch to have a pool of buffers that can be used for both input and output buffering. Still another possibility is to buffer on the input side, but allow the second or third cell in line to be switched, even if the first one cannot be.

Many other switch designs have been proposed and tried. These include time division switches using shared memory, buses or rings, as well as space division switches with one or more paths between each input and each output.

Some of these switches are discussed in (Ahmadi and Denzel, 1989; Anderson et al., 1993; Gopal et al., 1992; Pattavina, 1993; Rooholamini et al., 1994; and Zegura, 1993).

2.2.6. Some Implications of ATM for Distributed Systems

The availability of ATM networks at 155 Mbps, 622 Mbps, and potentially at 2.5 Gbps has some major implications for the design of distributed systems. For the most part, the effects are due primarily to the enormously high bandwidth suddenly available, rather than due to specific properties of ATM networks. The effects are most pronounced on wide-area distributed systems.

To start with, consider sending a 1-Mbit file across the United States and waiting for an acknowledgement that it has arrived correctly. The speed of light in copper wire or fiber optics is about $2/3$ the speed of light in vacuum, so it takes a bit about 15 msec to go across the US one way. At 64 Kbps, it takes about 15.6 sec to pump the bits out, so the additional 30 msec round-trip delay does not add much. At 622 Mbps, it takes $1/622$ of a second, or about 1.6 msec, to push the whole file out the door. In the best case, the reply can come back after 31.6 msec, during which time the line was idle for 30 msec, or 95 percent of the total. As speeds go up, the time-to-reply asymptotically approaches 30 msec, and the fraction of the available virtual circuit bandwidth that can be used approaches 0. For messages shorter than 1 Mbps, which are common in distributed systems, it is even worse. The conclusion is: For high-speed wide-area distributed systems, new protocols and system architectures will be needed to deal with the latency in many applications, especially interactive ones.

Another problem is flow control. Suppose that we have a truly large file, say a videotape consisting of 10 GB. The sender begins transmitting at 622 Mbps, and the data begin to roll in at the receiver. The receiver may not happen to have a 10 GB buffer handy, so it sends back a cell saying: STOP. By the time the STOP cell has gotten back to the sender, 30 msec later, almost 20 Mbits of data are under way. If most of these are lost due to inadequate buffer space, they will have to be transmitted again. Using a traditional sliding window protocol gets us back to the situation we just had, namely, if the sender is allowed to send only 1 Mbit and then has to wait for an acknowledgement, the virtual circuit is 95 percent idle. Alternatively, a large amount of buffering capacity can be put in the switches and adaptor boards, but at increased cost. Still another possibility is rate control, in which the sender and receiver agree in advance how many bits/sec the sender may transmit. Flow control and congestion control in ATM networks are discussed in (Eckberg, 1992; Hong and Suda, 1991; and Trajkovic and Golestani, 1992). A bibliography with over 250 references to performance in ATM networks is given in (Nikolaidis and Onvural, 1992).

A different approach to dealing with the now-huge 30 msec latency is to

send some bits, then stop the sending process and run something else while waiting for the reply. The trouble with this strategy is that computers are becoming so inexpensive, that for many applications, each process has its own computer, so there is nothing else to run. Wasting the CPU time is not important, since it is cheap, but it is clear that going from 64 Kbps to 622 Mbps has not bought a 10,000-fold gain in performance, even in communication-limited applications.

The effect of the transcontinental delay can show up in various ways. For example, if some application program in New York has to make 20 sequential requests from a server in California to get an answer, the 600-msec delay will be noticeable to the user, as people find delays above 200 msec annoying.

Alternatively, we could move the computation itself to the machine in California and let each user keystroke be sent as a separate cell across the country and come back to be displayed. Doing this will add 60 msec to each keystroke, which no one will notice. However, this reasoning quickly leads us to abandoning the idea of a distributed system and putting all the computing in one place, with remote users. In effect, we have built a big centralized timesharing system with just the users distributed.

One observation that does relate to specific properties of ATM is the fact that switches are permitted to drop cells if they get congested. Dropping even one cell probably means waiting for a timeout and having the whole packet be retransmitted. For services that need a uniform rate, such as playing music, this could be a problem. (Oddly enough, the ear is far more sensitive than the eye to irregular delivery.)

As a consequence of these and other problems, while high-speed networks in general and ATM in particular introduce new opportunities, taking advantage of them will not be simple. Considerable research will be needed before we know how to deal with them effectively.

2.3. THE CLIENT-SERVER MODEL

While ATM networks are going to be important in the future, for the moment they are too expensive for most applications, so let us go back to more conventional networking. At first glance, layered protocols along the OSI lines look like a fine way to organize a distributed system. In effect, a sender sets up a connection (a bit pipe) with the receiver, and then pumps the bits in, which arrive without error, in order, at the receiver. What could be wrong with this?

Plenty. To start with, look at Fig. 2-2. The existence of all those headers generates a considerable amount of overhead. Every time a message is sent it must be processed by about half a dozen layers, each one generating and adding a header on the way down or removing and examining a header on the way up. All of this work takes time. On wide-area networks, where the number of

bits/sec that can be sent is typically fairly low (often as little as 64K bits/sec), this overhead is not serious. The limiting factor is the capacity of the lines, and even with all the header manipulation, the CPUs are fast enough to keep the lines running at full speed. Thus a wide-area distributed system can probably use the OSI or TCP/IP protocols without any loss in (the already meager) performance. Aith ATM, even here serious problems may arise.

However, for a LAN-based distributed system, the protocol overhead is often substantial. So much CPU time is wasted running protocols that the effective throughput over the LAN is often only a fraction of what the LAN can do. As a consequence, most LAN-based distributed systems do not use layered protocols at all, or if they do, they use only a subset of the entire protocol stack.

In addition, the OSI model addresses only a small aspect of the problem—getting the bits from the sender to the receiver (and in the upper layers, what they mean). It does not say anything about how the distributed system should be structured. Something more is needed.

2.3.1. Clients and Servers

This something is often the client-server model that we introduced in the preceding chapter. The idea behind this model is to structure the operating system as a group of cooperating processes, called **servers**, that offer services to the users, called **clients**. The client and server machines normally all run the same microkernel, with both the clients and servers running as user processes, as we saw earlier. A machine may run a single process, or it may run multiple clients, multiple servers, or a mixture of the two.

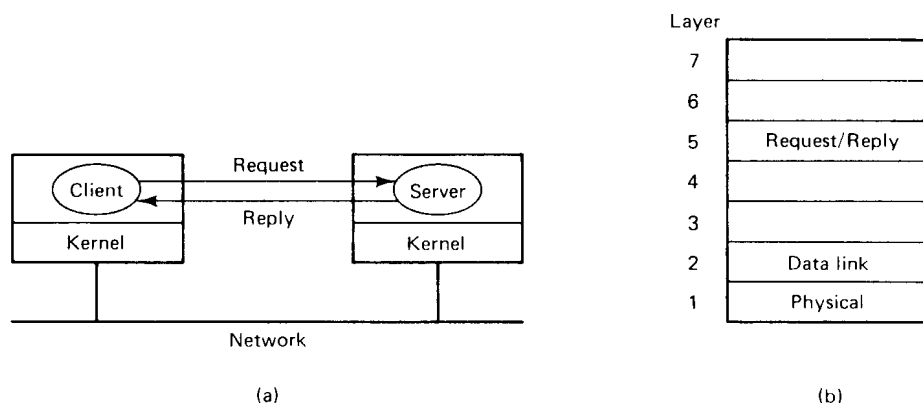


Fig. 2-7. The client-server model. Although all message passing is actually done by the kernels, this simplified form of drawing will be used when there is no ambiguity.

To avoid the considerable overhead of the connection-oriented protocols such as OSI or TCP/IP, the client server model is usually based on a simple, connectionless **request/reply protocol**. The client sends a request message to the server asking for some service (e.g., read a block of a file). The server does the work and returns the data requested or an error code indicating why the work could not be performed, as depicted in Fig. 2-7(a).

The primary advantage of Fig. 2-7(a) is the simplicity. The client sends a request and gets an answer. No connection has to be established before use or torn down afterward. The reply message serves as the acknowledgement to the request.

From the simplicity comes another advantage: efficiency. The protocol stack is shorter and thus more efficient. Assuming that all the machines are identical, only three levels of protocol are needed, as shown in Fig. 2-7(b). The physical and data link protocols take care of getting the packets from client to server and back. These are always handled by the hardware, for example, an Ethernet or token ring chip. No routing is needed and no connections are established, so layers 3 and 4 are not needed. Layer 5 is the request/reply protocol. It defines the set of legal requests and the set of legal replies to these requests. There is no session management because there are no sessions. The upper layers are not needed either.

Due to this simple structure, the communication services provided by the (micro)kernel can, for example, be reduced to two system calls, one for sending messages and one for receiving them. These system calls can be invoked through library procedures, say, *send(dest, &mptr)* and *receive(addr, &mptr)*. The former sends the message pointed to by *mptr* to a process identified by *dest* and causes the caller to be blocked until the message has been sent. The latter causes the caller to be blocked until a message arrives. When one does, the message is copied to the buffer pointed to by *mptr* and the caller is unblocked. The *addr* parameter specifies the address to which the receiver is listening. Many variants of these two procedures and their parameters are possible. We will discuss some of these later in this chapter.

2.3.2. An Example Client and Server

To provide more insight into how clients and servers work, in this section we will present an outline of a client and a file server in C. Both the client and the server need to share some definitions, so we will collect these into a file called *header.h*, which is shown in Fig. 2-8. Both the client and server include these using the

```
#include <header.h>
```

statement. This statement has the effect of causing a preprocessor to literally

insert the entire contents of *header.h* into the source program just before the compiler starts compiling the program.

```
/* Definitions needed by clients and servers. */
#define MAX_PATH    255      /* maximum length of a file name */
#define BUF_SIZE    1024    /* how much data to transfer at once */
#define FILE_SERVER 243     /* file server's network address */

/* Definitions of the allowed operations. */
#define CREATE      1       /* create a new file */
#define READ        2       /* read a piece of a file and return it */
#define WRITE       3       /* write a piece of a file */
#define DELETE      4       /* delete an existing file */

/* Error codes. */
#define OK          0       /* operation performed correctly */
#define E_BAD_OPCODE -1     /* unknown operation requested */
#define E_BAD_PARAM -2     /* error in a parameter */
#define E_IO        -3     /* disk error or other I/O error */

/* Definition of the message format. */
struct message {
    long source;           /* sender's identity */
    long dest;             /* receiver's identity */
    long opcode;           /* which operation: CREATE, READ, etc. */
    long count;            /* how many bytes to transfer */
    long offset;           /* where in file to start reading or writing */
    long extra1;           /* extra field */
    long extra2;           /* extra field */
    long result;           /* result of the operation reported here */
    char name[MAX_PATH];   /* name of the file being operated on */
    char data[BUF_SIZE];   /* data to be read or written */
};
```

Fig. 2-8. The *header.h* file used by the client and server.

Let us first take a look at *header.h*. It starts out by defining two constants, *MAX_PATH* and *BUF_SIZE*, that determine the size of two arrays needed in the message. The former tells how many characters a file name (i.e., a path name like */usr/ast/books/opsys/chapter1.t*) may contain. The latter fixes the amount of data that may be read or written in one operation by setting the buffer size. The next constant, *FILE_SERVER*, provides the network address of the file server so that clients can send messages to it.

The second group of constants defines the operation numbers. These are needed to ensure that the client and server agree on which code will represent a READ, which code will represent a WRITE, and so on. We have only shown four here, but in a real system there would normally be more.

Every reply contains a result code. If the operation succeeds, the result code often contains useful information (such as the number of bytes actually read). If there is no value to be returned (such as when a file is created), the value *OK* is

used. If the operation is unsuccessful for some reason, the result code tells why, using codes such as *E_BAD_OPCODE*, *E_BAD_PARAM*, and so on.

Finally, we come to the most important part of *header.h*, the definition of the message itself. In our example it is a structure with 10 fields. All requests from the client to the server use this format, as do all replies. In a real system, one would probably not have a fixed format message (because not all the fields are needed in all cases), but it makes the explanation simpler here. The *source* and *dest* fields identify the sender and receiver, respectively. The *opcode* field is one of the operations defined above, that is, *CREATE*, *READ*, *WRITE*, or *DELETE*. The *count* and *offset* fields are used for parameters, and two other fields, *extra1* and *extra2*, are defined to provide space for additional parameters in case the server is expanded in the future. The *result* field is not used for client-to-server requests but holds the result value for server-to-client replies. Finally, we have two arrays. The first, *name*, holds the name of the file being accessed. The second, *data*, holds the data sent back on a reply to *READ* or the data sent to the server on a *WRITE*.

Let us now look at the code, as outlined in Fig. 2-9. In (a) we have the server; in (b) we have the client. The server is straightforward. The main loop starts out by calling *receive* to get a request message. The first parameter identifies the caller by giving its address, and the second parameter points to a message buffer where the incoming message can be stored. The library procedure *receive* traps to the kernel to suspend the server until a message arrives. When one comes in, the server continues and dispatches on the opcode type. For each opcode, a different procedure is called. The incoming message and a buffer for the outgoing message are given as parameters. The procedure examines the incoming message, *m1*, and builds the reply in *m2*. It also returns a function value that is sent back in the *result* field. After the *send* has completed, the server goes back to the top of the loop to execute *receive* and wait for the next incoming message.

In Fig. 2-9(b) we have a procedure that copies a file using the server. Its body consists of a loop that reads one block from the source file and writes it to the destination file. The loop is repeated until the source file has been copied completely, as indicated by a zero or negative return code from the read.

The first part of the loop is concerned with building a message for the *READ* operation and sending it to the server. After the reply has been received, the second part of the loop is entered, which takes the data just received and sends it back to the server in the form of a *WRITE* to the destination file. The programs of Fig. 2-9 are just sketches of the code. Many details have been omitted. For example, the *do_xxx* procedures (the ones that actually do the work) are not shown, and no error checking is done. Still, the general idea of how a client and a server interact should be clear. In the following sections we will look at some of the issues that relate to clients and servers in more detail.

```

#include <header.h>
void main(void)
{
    struct message m1, m2;          /* incoming and outgoing messages */
    int r;                          /* result code */

    while (1) {                    /* server runs forever */
        receive(FILE_SERVER, &m1); /* block waiting for a message */
        switch(m1.opcode) {        /* dispatch on type of request */
            case CREATE:           r = do_create(&m1, &m2);      break;
            case READ:             r = do_read(&m1, &m2);        break;
            case WRITE:            r = do_write(&m1, &m2);        break;
            case DELETE:           r = do_delete(&m1, &m2);       break;
            default:               r = E_BAD_OPCODE;
        }
        m2.result = r;              /* return result to client */
        send(m1.source, &m2);      /* send reply */
    }
}                                  (a)

#include <header.h>
int copy(char *src, char *dst)     /* procedure to copy file using the server */
{
    struct message m1;             /* message buffer */
    long position;                 /* current file position */
    long client = 110;             /* client's address */

    initialize();                  /* prepare for execution */
    position = 0;
    do {
        /* Get a block of data from the source file. */
        m1.opcode = READ;          /* operation is a read */
        m1.offset = position;      /* current position in the file */
        m1.count = BUF_SIZE;      /* how many bytes to read */
        strcpy(&m1.name, src);     /* copy name of file to be read to message */
        send(FILE_SERVER, &m1);    /* send the message to the file server */
        receive(client, &m1);      /* block waiting for the reply */

        /* Write the data just received to the destination file. */
        m1.opcode = WRITE;         /* operation is a write */
        m1.offset = position;      /* current position in the file */
        m1.count = m1.result;      /* how many bytes to write */
        strcpy(&m1.name, dst);     /* copy name of file to be written to buf */
        send(FILE_SERVER, &m1);    /* send the message to the file server */
        receive(client, &m1);      /* block waiting for the reply */
        position += m1.result;     /* m1.result is number of bytes written */
    } while (m1.result > 0);        /* iterate until done */
    return(m1.result >= 0 ? OK : m1.result); /* return OK or error code */
}                                  (b)

```

Fig. 2-9. (a) A sample server. (b) A client procedure using that server to copy a file.

2.3.3. Addressing

In order for a client to send a message to a server, it must know the server's address. In the example of the preceding section, the server's address was simply hardwired into *header.h* as a constant. While this strategy might work in an especially simple system, usually a more sophisticated form of addressing is needed. In this section we will describe some issues concerning addressing.

In our example, the file server has been assigned a numerical address (243), but we have not really specified what this means. In particular, does it refer to a specific machine, or to a specific process? If it refers to a specific machine, the sending kernel can extract it from the message structure and use it as the hardware address for sending the packet to the server. All the sending kernel has to do then is build a frame using the 243 as the data link address and put the frame out on the LAN. The server's interface board will see the frame, recognize 243 as its own address, and accept it.

If there is only one process running on the destination machine, the kernel will know what to do with the incoming message—give it to the one and only process running there. However, what happens if there are several processes running on the destination machine? Which one gets the message? The kernel has no way of knowing. Consequently, a scheme that uses network addresses to identify processes means that only one process can run on each machine. While this limitation is not fatal, it is sometimes a serious restriction.

An alternative addressing system sends messages to processes rather than to machines. Although this method eliminates all ambiguity about who the real recipient is, it does introduce the problem of how processes are identified. One common scheme is to use two part names, specifying both a machine and a process number. Thus 243.4 or 4@243 or something similar designates process 4 on machine 243. The machine number is used by the kernel to get the message correctly delivered to the proper machine, and the process number is used by the kernel on that machine to determine which process the message is intended for. A nice feature of this approach is that every machine can number its processes starting at 0. No global coordination is needed because there is never any ambiguity between process 0 on machine 243 and process 0 on machine 199. The former is 243.0 and the latter is 199.0. This scheme is illustrated in Fig. 2-10(a).

A slight variation on this addressing scheme uses *machine.local-id* instead of *machine.process*. The *local-id* field is normally a randomly chosen 16-bit or 32-bit integer (or the next one in sequence). One process, typically a server, starts up by making a system call to tell the kernel that it wants to listen to *local-id*. Later, when a message comes in addressed to *machine.local_id*, the kernel knows which process to give the message to. Most communication in Berkeley UNIX, for example, uses this method, with 32-bit Internet addresses used for specifying machines and 16-bit numbers for the *local-id* fields.

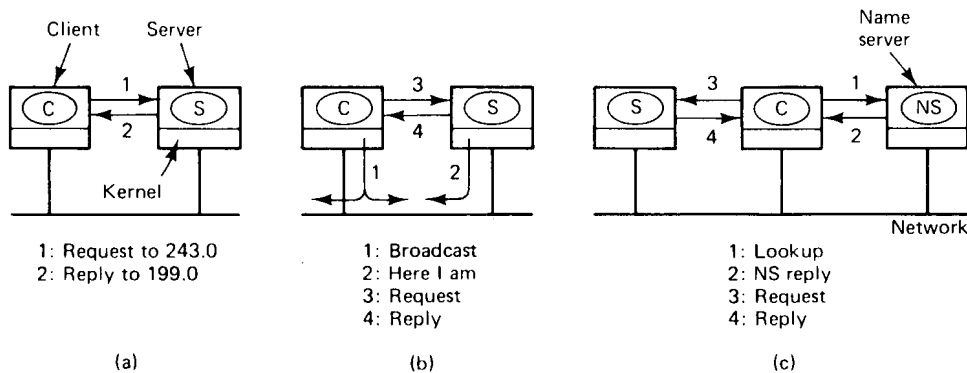


Fig. 2-10. (a) Machine.process addressing. (b) Process addressing with broadcasting. (c) Address lookup via a name server.

Nevertheless, *machine.process* addressing is far from ideal. Specifically, it is not transparent since the user is obviously aware of where the server is located, and transparency is one of the main goals of building a distributed system. To see why this matters, suppose that the file server normally runs on machine 243, but one day that machine is down. Machine 176 is available, but programs previously compiled using *header.h* all have the number 243 built into them, so they will not work if the server is unavailable. Clearly, this situation is undesirable.

An alternative approach is to assign each process a unique address that does not contain an embedded machine number. One way to achieve this goal is to have a centralized process address allocator that simply maintains a counter. Upon receiving a request for an address, it simply returns the current value of the counter and then increments it by one. The disadvantage of this scheme is that centralized components like this do not scale to large systems and thus should be avoided.

Yet another method for assigning process identifiers is to let each process pick its own identifier from a large, sparse address space, such as the space of 64-bit binary integers. The probability of two processes picking the same number is tiny, and the system scales well. However, here, too, there is a problem: How does the sending kernel know what machine to send the message to? On a LAN that supports broadcasting, the sender can broadcast a special **locate packet** containing the address of the destination process. Because it is a broadcast packet, it will be received by all machines on the network. All the kernels check to see if the address is theirs, and if so, send back a **here I am** message giving their network address (machine number). The sending kernel then uses

this address, and furthermore, caches it, to avoid broadcasting the next time the server is needed. This method is shown in Fig. 2-10(b).

Although this scheme is transparent, even with caching, the broadcasting puts extra load on the system. This extra load can be avoided by providing an extra machine to map high-level (i.e., ASCII) service names to machine addresses, as shown in Fig. 2-10(c). When this system is employed, processes such as servers are referred to by ASCII strings, and it is these strings that are embedded in programs, not binary machine or process numbers. Every time a client runs, on the first attempt to use a server, the client sends a query message to a special mapping server, often called a **name server**, asking it for the machine number where the server is currently located. Once this address has been obtained, the request can be sent directly. As in the previous case, addresses can be cached.

In summary, we have the following methods for addressing processes:

1. Hardwire *machine.number* into client code.
2. Let processes pick random addresses; locate them by broadcasting.
3. Put ASCII server names in clients; look them up at run time.

Each of these has problems. The first one is not transparent, the second one generates extra load on the system, and the third one requires a centralized component, the name server. Of course, the name server can be replicated, but doing so introduces the problems associated with keeping them consistent.

A completely different approach is to use special hardware. Let processes pick random addresses. However, instead of locating them by broadcasting, the network interface chips have to be designed to allow processes to store process addresses in them. Frames would then use process addresses instead of machine addresses. As each frame came by, the network interface chip would simply examine the frame to see if the destination process was on its machine. If so, the frame would be accepted; otherwise, it would not be.

2.3.4. Blocking versus Nonblocking Primitives

The message-passing primitives we have described so far are what are called **blocking primitives** (sometimes called **synchronous primitives**). When a process calls *send* it specifies a destination and a buffer to send to that destination. While the message is being sent, the sending process is blocked (i.e., suspended). The instruction following the call to *send* is not executed until the message has been completely sent, as shown in Fig. 2-11(a). Similarly, a call to *receive* does not return control until a message has actually been received and put in the message buffer pointed to by the parameter. The process remains

suspended in *receive* until a message arrives, even if it takes hours. In some systems, the receiver can specify from whom it wishes to receive, in which case it remains blocked until a message from that sender arrives.

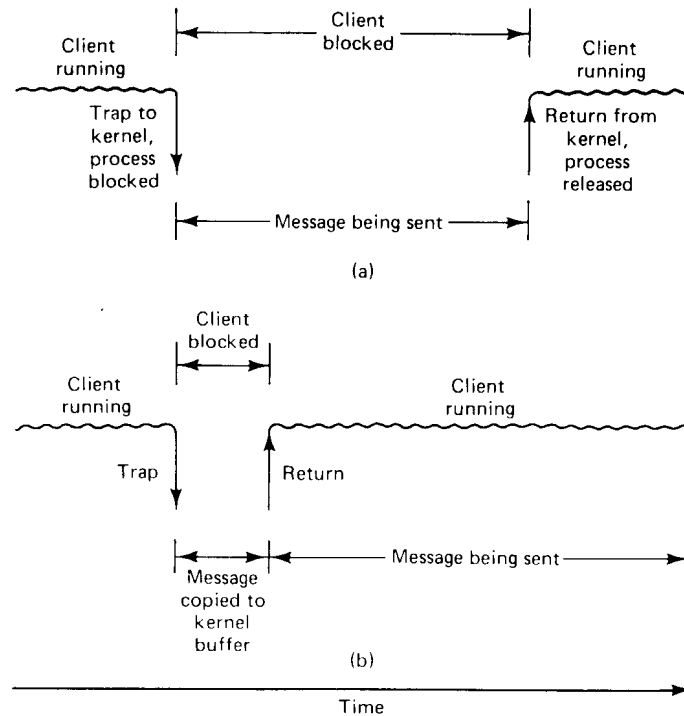


Fig. 2-11. (a) A blocking send primitive. (b) A nonblocking send primitive.

An alternative to blocking primitives are **nonblocking primitives** (sometimes called **asynchronous primitives**). If *send* is nonblocking, it returns control to the caller immediately, before the message is sent. The advantage of this scheme is that the sending process can continue computing in parallel with the message transmission, instead of having the CPU go idle (assuming no other process is runnable). The choice between blocking and nonblocking primitives is normally made by the system designers (i.e., either one primitive is available or the other), although in a few systems both are available and users can choose their favorite.

However, the performance advantage offered by nonblocking primitives is offset by a serious disadvantage: the sender cannot modify the message buffer until the message has been sent. The consequences of the process overwriting the message during transmission are too horrible to contemplate. Worse yet, the sending process has no idea of when the transmission is done, so it never knows when it is safe to reuse the buffer. It can hardly avoid touching it forever.

There are two possible ways out. The first solution is to have the kernel copy the message to an internal kernel buffer and then allow the process to continue, as shown in Fig. 2-11(b). From the sender's point of view, this scheme is the same as a blocking call: as soon as it gets control back, it is free to reuse the buffer. Of course, the message will not yet have been sent, but the sender is not hindered by this fact. The disadvantage of this method is that every outgoing message has to be copied from user space to kernel space. With many network interfaces, the message will have to be copied to a hardware transmission buffer later anyway, so the first copy is essentially wasted. The extra copy can reduce the performance of the system considerably.

The second solution is to interrupt the sender when the message has been sent to inform it that the buffer is once again available. No copy is required here, which saves time, but user-level interrupts make programming tricky, difficult, and subject to race conditions, which makes them irreproducible. Most experts agree that although this method is highly efficient and allows the most parallelism, the disadvantages greatly outweigh the advantages: programs based on interrupts are difficult to write correctly and nearly impossible to debug when they are wrong.

Sometimes the interrupt can be disguised by starting up a new thread of control (to discussed in Chap. 4) within the sender's address space. Although this is somewhat cleaner than a raw interrupt, it is still far more complicated than synchronous communication. If only a single thread of control is available, the choices come down to:

1. Blocking send (CPU idle during message transmission).
2. Nonblocking send with copy (CPU time wasted for the extra copy).
3. Nonblocking send with interrupt (makes programming difficult).

Under normal conditions, the first choice is the best. It does not maximize the parallelism, but is simple to understand and simple to implement. It also does not require any kernel buffers to manage. Furthermore, as can be seen from comparing Fig. 2-11(a) to Fig. 2-11(b), the message will usually be out the door faster if no copy is required. On the other hand, if overlapping processing and transmission are essential for some application, a nonblocking send with copying is the best choice.

For the record, we would like to point out that some authors use a different criterion to distinguish synchronous from asynchronous primitives (Andrews, 1991). In our view, the essential difference between a synchronous primitive and an asynchronous one is whether the sender can reuse the message buffer immediately after getting control back without fear of messing up the *send*. When the message actually gets to the receiver is irrelevant.